

DESIGN AND DEVELOPMENT OF EMBEDDED CONTROL SYSTEM FOR A
SUPERVISED TELEOPERATION 6 DOF SERIAL CHAIN MANIPULATOR

BY

OBAID TANVEER (02-3-1-062-2020)

MUHAMMAD SHERAZ (02-3-1-053-2020)

MUHAMMAD HUZAIFA (02-3-1-042-2020)

THESIS SUBMITTED TO THE FACULTY OF ENGINEERING AT PIEAS IN
PARTIAL FULFILLMENT OF REQUIREMENTS FOR THE DEGREE OF BS
ELECTRICAL ENGINEERING.



DEPARTMENT OF ELECTRICAL ENGINEERING,
PAKISTAN INSTITUTE OF ENGINEERING AND APPLIED SCIENCES,
NILORE, ISLAMABAD 45650, PAKISTAN

JUNE, 2024

Department of Electrical Engineering

Pakistan Institute of Engineering and Applied Sciences (PIEAS)

Nilore, Islamabad 45650, Pakistan

Declaration of Originality

We hereby declare that the work contained in this thesis and the intellectual content therein are the product of our own research. This thesis has not been previously published in any form nor does it contain any verbatim of the published resources which could be treated as infringement of the international copyright law. We also declare that we do understand the terms ‘copyright’ and ‘plagiarism,’ and that in case of any copyright violation or plagiarism found in this work, we will be held fully responsible of the consequences of any such violation.

Signature: _____

Name: XYZ

Signature: _____

Name: ABC

Signature: _____

Name: PQR

Date: Month Day, Year

Place: PIEAS

Certificate of Approval

This is to certify that the work contained in this thesis entitled

“Design and Development of Embedded Control System for a
Supervised Teleoperation 6 DOF Serial Chain Manipulator ”

carried out by

Obaid Tanveer (02-3-1-062-2020)

Muhammad Sheraz (02-3-1-053-2020)

Muhammad Huzaifa (02-3-1-042-2020)

*It is fully adequate, in scope and quality, for the degree of BS
Electrical Engineering from Pakistan Institute of Engineering and
Applied Sciences (PIEAS).*

Approved By:

Signature: _____

Co-Supervisor: XYZ

Signature: _____ Signature: _____

Name: XYZ

Name: XYZ

Academic Supervisor

Industrial supervisor

Verified By:

Signature: _____

Head, Department of Electrical Engineering

Stamp:

To parents, teachers, and muslims of Palestine

Acknowledgements

All thanks to Almighty Allah for He has blessed us with countless bounties. We would like to express our deepest gratitude to our honorable supervisor Dr. Nasir Rehman Jadoon and Co-supervisor Mr. Tanveer Abbas for guiding and supporting us throughout the project work, always showing patience, and sharing their knowledge. They helped us, whenever and wherever, it was required. Furthermore, they have been very regular in checking our progress on the project.

We would like to acknowledge other faculty members as well. They have remained very kind in teaching and guiding us. All of them are very nice, supportive, and best teachers. Mr. kaleem of Robotics Lab, DEE Department has been very kind throughout the project. We appreciate his efforts and guidance towards us for finance related issues.

Finally, we would like to thank our parents for their motivation and never ending support. It is because of their sacrifices, and prayers that we have reached the point, where we are.

Contents

List of Figures	xi
1 Introduction	2
1.1 Background	2
1.2 Motivation	3
1.3 Methodology	4
1.4 Applications:	6
2 Literature Review	8
2.1 Robotics	8
2.1.1 Robot Joints and Degrees of Freedom	8
2.1.2 Coordinate Transformations in Robotics	9
2.1.3 Robot Kinematics	9
2.1.4 Robot Dynamics	9
2.1.5 Sensors and Perception in Robotics	10
2.1.6 DH Parameters	10
2.1.7 Forward Kinematics	10
2.1.7.1 Mathematical Formulation	10
2.1.7.2 Importance and Applications	11
2.1.8 Inverse Kinematics	11
2.1.8.1 Mathematical Formulation	11
2.1.8.2 Importance and Applications	12
2.1.9 Trajectory Planning	12

2.2	Embedded Systems in Robotics	13
2.2.1	Hardware Architecture for Robotics	13
2.2.2	Software Frameworks in Robotics	13
2.2.3	Control Algorithms for Embedded Systems	14
2.2.4	Real-time Operating Systems (RTOS) in Robotics	14
2.2.5	Sensor Fusion and Data Processing	14
2.2.6	Power Management in Embedded Systems	15
2.2.7	Security Considerations in Embedded Systems	15
2.2.8	Fault Tolerance and Redundancy in Robotics	15
2.3	Teleoperation	16
2.3.1	Direct Teleoperation	16
2.3.2	Supervisory teleoperation	16
2.3.3	Multimodal teleoperation	17
2.3.4	Telepresence	17
2.3.5	Shared Control	18
2.4	Master-Slave Control Algorithm	18
2.4.1	Control Strategies	19
2.4.2	Working Agenda	19
2.4.3	Applications	19
2.5	Communication Protocols in embedded systems	20
2.5.1	Inter-Integrated Circuit (I2C)	21
2.5.2	Serial Peripheral Interface (SPI)	21
2.5.3	Controller Area Network	22
2.5.4	Bluetooth and Wi-Fi:	23
2.5.5	Modbus:	24
2.5.6	Universal Asynchronous Receiver-Transmitter (UART): . . .	24
3	Preliminaries	25
3.1	Software	25
3.1.1	STM32CubeMX: Configuring STM32 Microcontrollers . . .	25

3.1.2	Keil μ Vision	26
3.1.3	FreeRTOS	27
3.1.4	SolidWorks	30
3.1.5	V-REP/coppeliasim	30
3.2	Microcontroller	31
3.2.1	Nucleo - F767ZI	31
3.2.2	STM32F103C8T6 Blue Pill	33
3.3	Communications	35
3.3.1	CAN Bus - Controller Area Network	35
3.4	Limit switches	37
3.4.1	Sensor Module Specifications:	37
3.4.2	Limit Switch Functionality:	38
4	Kinematic Modeling	40
4.1	CAD Model of the robot	40
4.2	Forward Kinematics	41
4.2.1	Denavit-Hartenberg Parameters	41
4.3	Inverse kinematics	44
5	Simulations	46
5.1	Forward kinematics	46
5.2	Inverse Kinametics	47
5.3	Work Space	49
6	Motor control	51
6.1	DC Motor Analysis	51
6.2	Position and Velocity Control	52
6.3	Real-Time Sensor Feedback	52
6.4	Feedback-Control Mechanisms	52
6.5	Motor Control Techniques	53
6.5.1	Trapezoidal Velocity Control:	53

6.5.2	Cubic Polynomial with Time Slicing:	54
6.5.3	Proportional-Integral-Derivative (PID) Control:	55
7	Hardware implementation	57
7.1	Hardware Components	57
7.2	Microprocessor Integration	58
7.2.1	Master Controller:	59
7.2.2	Slave Controllers:	59
7.3	Motor Control	60
8	Conclusion	62
8.1	Summary	62
8.2	Future Work	63
8.3	Sustainable Development Goals	63
	References	65

List of Figures

2.1	Master-Slavecontrol Architecture	20
3.1	STM32 Nucleo-144 development board	33
3.2	STM32 Bluepill development board	35
3.3	MCP2551 CAN module for communication between master and slaves	37
3.4	U-shaped sensor module used as a limit switch	38
4.1	Cad modeol of the extended -DOF robot	40
4.2	Extended 5 DOF robot	41
4.3	Link Co-ordinate diagram	42
5.1	Forward Kinemetics	46
5.2	Verification of forward kinematics from simulation	47
5.3	Verification of Inverse kinematics from simulation	48
5.4	giving angles to the robot	48
5.5	giving angles to the robot	48
5.6	Wrokspace envelope of the robot	49
5.7	Front view of the workspace	49
5.8	Side view of the workspace	50
5.9	Top view of the workspace	50
6.1	Position, Velocity, and Accelerations in a trapezoidal velocity mo- tion profile	54
7.1	Overall Hardware circuit	60

Abstract

This thesis explores the process of designing a sophisticated control system that enables supervised and accurate teleoperation of a six-degree-of-freedom (DOF) serial chain manipulator. This project requires a thorough design and development process, as well as a sophisticated understanding of mechanical engineering, robotics, and control systems. Beyond traditional models, the work incorporates state-of-the-art technology with an emphasis on improved controllability and versatility in a range of applications. By pushing the boundaries of teleoperation systems, the thesis aspires to contribute considerably to the field, delivering a reliable and efficient solution uniquely tailored for the problems given by complicated manipulator configurations. This one-of-a-kind thesis is evidence of the dedication to creativity, pushing the boundaries of technology to create sophisticated and responsive control systems in complex robotic designs.

Chapter 1

Introduction

The introduction chapter of this thesis begins by providing a comprehensive background of robotics and the critical role manipulator systems play in various industries. It presents the problem statement with great care, highlighting the need for accuracy and supervised control with a 6-DOF serial chain manipulator. There is a clear explanation of the research's goals together with a consideration of the study's limitations, importance, and scope. Crucially, it explores the motivation behind choosing this particular subject. In addition to outlining the general framework of the thesis, the chapter clarifies the selected approach and tools and poses relevant research questions and hypotheses.

1.1 Background

The field of robotics has developed into one that is revolutionary, with applications in fields including industry, healthcare, and exploration. Manipulator systems are essential to this environment because they allow for accurate and controlled movements in a variety of operational settings. However, there are inherent obstacles in realizing effective teleoperation, especially for complicated setups such as a 6-degree-of-freedom (DOF) serial chain manipulator. The requirement for increased precision and the capacity to monitor and intervene in real-time operations gives rise to the need for sophisticated teleoperation control systems. These needs are

frequently unmet by current approaches, which calls for a focused investigation into the design and development of a system that satisfies the requirements of modern robotic applications. This study is a pathfinder in the booming field of robotics and automation, ready to rekindle the lively debate about these innovative technologies. In an attempt to understand the complex dance between supervision and precision, our research explores the unexplored realm of teleoperation control with an overflowing sense of curiosity. A master of the gaps and constraints resonating across the current story, the project stands as a vast symphony of robotic orchestration. By highlighting these details, we hope to both build upon the current conversation and push the envelope of what is possible in the fascinating field of robotics.

1.2 Motivation

In the race for technical advancement and innovation, the motivation behind this project lies in addressing critical gaps in the current landscape of robotics and automation. This thesis aims to address the following critical challenges in the field of robotics

- **Precision Control Challenges:**When it comes to handling the complex movements of a 6-degree-of-freedom (DOF) serial chain manipulator, existing control methodologies fall short of the requirements for precision control. The manipulator's application in different industries is limited due to its inability to perform complex jobs accurately when precision control is lacking.
- **Supervised Teleoperation Shortcomings:**Strong mechanisms for real-time supervision and intervention are absent from current teleoperation control systems, which makes it difficult to monitor and adjust to changing operational conditions. A lack of sufficient supervision capabilities limits the

flexibility of manipulator systems and limits their use in tasks that need to be adjusted and monitored in real time.

- **Technological Gaps and Integration Issues:** The swift development of manipulator systems necessitates sophisticated control systems capable of smoothly incorporating state-of-the-art technologies. The inability of control systems to adjust to a variety of applications and operating conditions is a consequence of technology gaps that restrict the flexibility of manipulator configurations.
- **Lack of Adaptive Control Systems:** Complex manipulator designs provide dynamic issues that cannot be adequately addressed by conventional control approaches. The lack of adaptive control makes it more difficult for the manipulator to operate in a variety of applications, which reduces its usefulness in a quickly evolving technological environment.
- **Contributions to the Discourse:** There is an absence of thorough understandings and approaches in the current discourse that go beyond the goal of optimizing operational effectiveness in teleoperation control systems. This disparity keeps the field from moving forward in the direction of revolutionary solutions that can influence the stories that control systems in complex robotic designs tell in the future.

This project's inspiration originates from tackling these complex issues, which push it in the direction of revolutionary solutions that seek to expand the boundaries of robotics and teleoperation control systems.

1.3 Methodology

Following is the methodology that is adopted:

- Conduct an extensive review of existing literature on teleoperation control systems, focusing on precision methodologies and supervised frameworks.

Identify key technologies, algorithms, and methodologies utilized in similar projects to inform the design process.

- Describe the particular difficulties and needs related to supervised teleoperation and precision control within the framework of a six degree-of-freedom serial chain manipulator. Work together with professionals in the field to get knowledge about practical uses and operational situations.
- Create a thorough system architecture that outlines the elements, relationships, and features needed for accurate control and in-the-moment oversight. Assess possible technologies to make sure they work with the manipulator design, such as sensors and communication protocols.
- Create and put into practice control algorithms specifically designed to accomplish the 6 DOF serial chain manipulator's precise movements. Incorporate control mechanisms that are adaptable to improve responsiveness and flexibility in a range of operating situations.
- Create a framework for supervised teleoperation that includes tools for real-time monitoring and intervention. Establish user-friendly interface mechanisms that facilitate efficient communication and control between the manipulator and the operator.
- Investigate and incorporate cutting-edge technologies to improve the accuracy and flexibility of the control system, such as computer vision or machine learning. Check that integrated technologies are seamlessly compatible with the overall architecture of the system.
- Make extensive use of simulation environments to thoroughly evaluate the control system under different conditions. Evaluate the system's functionality in the actual world while resolving any issues and optimizing the algorithms.

- Quantitatively evaluate the precision, responsiveness, and adaptability of the developed control system. Compare the results with benchmarks established in the literature and industry standards.
- Implement feedback loops for iterative refinement, allowing for continuous improvement based on the outcomes of simulations and real-world testing. Ensure the adaptability of the control system to evolving technological landscapes.

1.4 Applications:

This project finds application in different fields including:

- **Manufacturing and Assembly:** Enable precise control for intricate assembly tasks, enhancing efficiency in manufacturing processes. Facilitate remote supervision in manufacturing plants, reducing the need for physical presence in hazardous environments.
- **Medical Robotics:** Enhance surgical procedures by providing surgeons with precise control over robotic arms during minimally invasive surgeries. Enable teleoperated medical interventions, expanding access to specialized procedures in remote locations.
- **Search and Rescue Operations:** Navigate complex and hazardous terrains with enhanced precision, allowing for more effective search and rescue missions. Provide real-time oversight for operators in situations where direct human intervention is challenging. Enable teleoperated rehabilitation devices for patients, providing personalized and supervised therapy sessions. Enhance telemedicine applications by offering precise control over medical instruments for remote diagnostic procedures.
- **Industrial Inspections:** Conduct detailed inspections of infrastructure, such as pipelines and power lines, using teleoperated manipulators for en-

hanced visual assessments. Enable remote operators to oversee and control inspections in challenging environments.

- **Education and Research:** Serve as a valuable tool in educational settings, allowing students to gain hands-on experience in robotics and teleoperation. Facilitate research experiments by providing a platform for testing and validating control algorithms in diverse scenarios.
- **Defense and Security:** Deploy teleoperated manipulators for bomb disposal and hazardous material handling in defense and security operations. Provide remote surveillance and intervention capabilities in military applications.
- **Agricultural Automation:** Enable precise control for agricultural tasks, such as harvesting and planting, optimizing the efficiency of farming operations. Facilitate remote monitoring and control of agricultural equipment in large-scale farming environments.

Supervised Teleoperation Control System for a 6 DOF Serial Chain Manipulator's adaptability presents a wide range of opportunities, demonstrating its capacity to improve robotic system capabilities in a variety of applications and transform operations across sectors.

Chapter 2

Literature Review

2.1 Robotics

Robotics, a field bridging various disciplines like engineering and computer science, is crucial for modern technology. This section aims to explore the core ideas that shape robotic systems. Here, we'll uncover the basics behind how robots work. From understanding their movements and controls to how they see and interact with the world, we'll dive into the essentials. We'll cover everything from the theory guiding robot actions to how these ideas are applied in real-world situations. Plus, we'll touch on how robots and people can work together and the bigger picture of where robotics might take us. This section is a guide to the world of robotics. It aims to explain the important concepts in a way that's easy to understand, giving insight into how robots impact our lives and what the future might hold.

2.1.1 Robot Joints and Degrees of Freedom

In robotics, joints are the pivotal points allowing movement within a robotic system. Revolute joints permit rotational motion, akin to human joints, facilitating versatile and dynamic actions. Prismatic joints, alternatively, enable linear motion, extending or retracting parts of the robot. The Degrees of Freedom (DoF) quantify the robot's flexibility in terms of independent movements. For example, a robot with six DoF can perform translations and rotations independently along

six different axes, providing a broad range of motion crucial for intricate tasks in various applications.

2.1.2 *Coordinate Transformations in Robotics*

Coordinate transformations are the mathematical mechanisms governing how robots interpret and navigate space. Using homogeneous transformations, robots can seamlessly transition between different coordinate frames, facilitating precise spatial understanding. This involves translation and rotation matrices, allowing robots to represent their position and orientation accurately. Coordinate transformations are fundamental for tasks like localization, mapping, and manipulating objects within a three-dimensional environment.

2.1.3 *Robot Kinematics*

Robot kinematics explores the motion of robotic systems without delving into the forces involved. Forward kinematics calculates the position and orientation of the robot's end-effector based on joint angles, providing insights into its reachable workspace. Inverse kinematics, conversely, determines the required joint angles to achieve a desired end-effector position, crucial for precise control and task execution. This subsection is foundational for understanding how robots move and position themselves in space.

2.1.4 *Robot Dynamics*

Robot dynamics is the study of forces and torques influencing a robot's motion. Utilizing formulations like Newton-Euler and Lagrange methods, this field delves into the intricate relationships between external forces, joint torques, and resulting robotic movements. A comprehensive understanding of robot dynamics is essential for designing control strategies that ensure stability, efficiency, and safety across various applications, from manufacturing to complex and adaptive robotic systems.

2.1.5 Sensors and Perception in Robotics

Sensors serve as the sensory perception mechanisms for robots, allowing them to gather information from their environment. Vision sensors capture visual data, proximity sensors detect nearby objects, and IMUs measure orientation and acceleration. Sensor fusion techniques integrate data from various sensors, enhancing the robot's ability to understand and navigate its surroundings. Effective sensor utilization is critical for decision-making, obstacle detection, and autonomous operation in diverse robotic applications.

2.1.6 DH Parameters

Denavit-Hartenberg (DH) parameters are a standardized framework for representing the geometry and kinematics of robotic manipulators. These parameters define the spatial relationship between consecutive joints, simplifying the mathematical modeling of complex robotic systems. Understanding DH parameters is essential for designing and controlling robotic arms efficiently.

2.1.7 Forward Kinematics

Forward kinematics in robotics is the process of determining the position and orientation of the end-effector (or tool) of a robotic system based on the values of its joint parameters. It establishes the mapping from the joint space to the Cartesian space, providing information on how the robot moves and where its end-effector is located in three-dimensional space.

2.1.7.1 Mathematical Formulation

The forward kinematics problem involves creating a transformation matrix that represents the cumulative effect of all the joint movements. Denoting the joint angles or positions as variables, the transformation matrix is typically constructed by multiplying individual transformation matrices associated with each joint. The

final matrix describes the complete transformation from the robot's base to its end-effector.

2.1.7.2 Importance and Applications

- **Motion Planning:** Forward kinematics is integral to planning the motion of a robot. Engineers use this information to design trajectories and paths for the robot to follow, ensuring efficient and accurate movement.
- **Simulation and Visualization:** Forward kinematics is crucial for simulating and visualizing the robot's movements in a virtual environment. This aids in validating designs, predicting behavior, and optimizing robotic systems.
- **Robot Design:** Engineers leverage forward kinematics to design robotic systems with specific kinematic structures. Understanding how joint movements affect the end-effector's position informs the design process.

2.1.8 Inverse Kinematics

Inverse kinematics is the reverse process of forward kinematics. It involves determining the values of the joint parameters that will position the end-effector at a desired location and orientation in space. In essence, it answers the question: "Given a desired end-effector pose, what should be the joint angles or positions?" Inverse kinematics is crucial for achieving precise control over a robotic system. It allows engineers to specify the desired position and orientation of the end-effector, and the robot can then calculate the necessary joint angles to attain that configuration.

2.1.8.1 Mathematical Formulation

Solving the inverse kinematics problem often requires finding the roots of complex trigonometric equations or using numerical methods. Analytical solutions

are feasible for simpler robot geometries, but for more complex robotic systems, iterative methods like the Newton-Raphson method or optimization techniques may be employed.

2.1.8.2 Importance and Applications

- **Precise Control:** Inverse kinematics is crucial for achieving precise control over a robotic system. It allows engineers to specify the desired position and orientation of the end-effector, and the robot can then calculate the necessary joint angles to attain that configuration.
- **Robot Programming:** When programming a robot for specific tasks, inverse kinematics simplifies the process. Engineers can focus on specifying the desired end-effector poses, and the robot can autonomously calculate the required joint movements.
- **Simulation and Virtual Prototyping:** Inverse kinematics is vital for simulating and prototyping robotic systems virtually. Engineers can test different scenarios and task configurations before implementing them on the physical robot, ensuring efficiency and reducing the risk of errors.

2.1.9 Trajectory Planning

Trajectory planning involves generating optimal paths for a robot to follow. This encompasses advanced algorithms that consider factors like obstacle avoidance, joint constraints, and dynamic limitations. Comprehensive trajectory planning ensures smooth, efficient, and collision-free robotic movements, making it a critical aspect of robotic control in diverse applications.

2.2 Embedded Systems in Robotics

Embedded systems are the foundational backbone of modern robotics, integrating hardware, software, control algorithms, and frameworks to orchestrate the functionality and performance of robotic platforms.

2.2.1 *Hardware Architecture for Robotics*

The hardware architecture of embedded systems in robotics comprises intricate designs tailored to specific computational demands. It includes microcontrollers or microprocessors intricately integrated with sensors, actuators, and specialized hardware modules. These designs emphasize signal processing efficiency, real-time data acquisition, power management, and integration of diverse components, shaping the system's responsiveness and computational capabilities across varied operational scenarios. For instance, custom hardware designs can optimize energy consumption, enhance computational throughput, and ensure reliable performance in real-time applications requiring precise control.

2.2.2 *Software Frameworks in Robotics*

Robotic frameworks, exemplified by ROS (Robot Operating System), establish an encompassing software infrastructure that facilitates seamless hardware abstraction, device communication, and high-level task execution. They incorporate standardized libraries, middleware, and tools enabling modularity and interoperability across diverse robotic systems. These frameworks enable efficient management of devices, providing a standardized platform for developers to build and deploy complex robotic applications. Additionally, they facilitate rapid prototyping, system integration, and facilitate collaborative development efforts within the robotics community.

2.2.3 Control Algorithms for Embedded Systems

Control algorithms serve as the neural network governing robotic behavior, stability, and task execution. PID controllers, state-space models, and model-based techniques dictate motion, stability, and accurate task execution. These algorithms are crucial for optimizing task execution, ensuring accuracy within real-time constraints, and adapting to dynamic environmental changes. For instance, advanced control algorithms can facilitate adaptive behavior, enabling robots to navigate complex environments, interact with objects, and accomplish tasks with precision and reliability.

2.2.4 Real-time Operating Systems (RTOS) in Robotics

RTOS tailored for robotics, such as FreeRTOS, QNX, or VxWorks, offer deterministic execution, providing robust task scheduling, efficient resource allocation, and precise synchronization. These systems ensure timely task execution critical for robotics applications, guaranteeing precise control, minimal latency, and enhanced system reliability. For instance, in safety-critical applications like autonomous vehicles or surgical robotics, RTOS ensures prompt response times, enabling the execution of critical tasks within strict time constraints while ensuring system stability and safety.

2.2.5 Sensor Fusion and Data Processing

Sensor fusion techniques amalgamate data from diverse sensors like cameras, LiDAR, and IMUs to enhance perception accuracy and reliability in robotic systems. Sophisticated algorithms, such as Kalman filters or Bayesian methods, synthesize sensor data, enabling a comprehensive understanding of the environment and facilitating informed decision-making in real-time robotic applications. For instance, sensor fusion can enable robots to navigate unpredictable environments, detect obstacles, and accurately estimate their position and orientation, enhancing their adaptability and reliability.

2.2.6 Power Management in Embedded Systems

Power management strategies encompass low-power modes, efficient voltage regulation, and energy-efficient designs that optimize energy consumption, extending the operational lifespan of embedded systems. These strategies are pivotal in ensuring sustained operation and reliability of robotic systems, especially in resource-constrained or remote environments. For instance, efficient power management systems can optimize energy consumption, enabling robots to operate for extended periods without frequent recharging, thus enhancing their autonomy and reliability in remote applications.

2.2.7 Security Considerations in Embedded Systems

Security measures, including secure boot mechanisms, cryptographic protocols, and intrusion detection systems, safeguard embedded systems against cyber threats. Implementing robust security measures is imperative to mitigate potential vulnerabilities, protecting the system's integrity, confidentiality, and preventing unauthorized access or manipulation. For example, security measures in embedded systems prevent unauthorized access to critical components, ensuring that robots operate securely and reliably, especially in environments where data privacy and system integrity are paramount, such as healthcare or defense applications.

2.2.8 Fault Tolerance and Redundancy in Robotics

Implementing fault tolerance mechanisms, redundancy strategies, and fault detection mechanisms bolster system reliability, minimizing the impact of potential failures. Hardware redundancy, fault isolation, and predictive maintenance strategies enhance system resilience, ensuring continuous operation and minimizing downtimes in critical robotic applications. For instance, fault tolerance mechanisms enable robots to continue functioning despite component failures, ensuring system

reliability and minimizing disruptions in critical applications such as manufacturing or space exploration.

2.3 Teleoperation

Teleoperation in robotics is a technique where a human operator remotely controls a robotic system using a designated interface. This method finds application in various domains such as search and rescue operations, space exploration, medical surgery, manufacturing, and unmanned aerial vehicles. The control interface, which can range from simple joysticks to advanced haptic devices, serves as a crucial link between the human operator and the robotic platform. Challenges include addressing latency issues, providing effective feedback to the operator, and ensuring the security of communication channels. Despite these challenges, teleoperation significantly extends the reach of human capabilities, allowing for operations in hazardous environments

2.3.1 *Direct Teleoperation*

Direct teleoperation is a form of human-robot interaction where the operator directly controls the robot's movements in real-time. The control commands from the operator are transmitted to the robot, and the robot executes these commands without significant autonomous decision-making. This method is often used in scenarios where the operator requires immediate and precise control, such as in surgical procedures, precise manufacturing tasks, or when navigating through complex environments.

2.3.2 *Supervisory teleoperation*

Supervisory teleoperation involves a higher level of autonomy for the robot, with the human operator overseeing and providing high-level guidance. In this mode, the robot has a certain degree of autonomy in executing tasks, and the operator intervenes when necessary. Supervisory teleoperation is commonly applied in situ-

ations where the robot needs to adapt to changing conditions or handle unforeseen obstacles. Examples include search and rescue missions, where the robot navigates complex terrain under human guidance.

2.3.3 *Multimodal teleoperation*

Multimodal teleoperation combines various modes of communication and control interfaces to enhance the overall teleoperation experience. It integrates multiple input sources such as gestures, voice commands, haptic feedback, and visual information. This approach aims to provide a more natural and intuitive interaction between the human operator and the robot. Multimodal teleoperation is advantageous in scenarios where the operator needs to convey complex commands or receive diverse sensory feedback for effective control. For instance, a robot may respond to both spoken commands and hand gestures, offering a versatile and adaptive control system

2.3.4 *Telepresence*

allows you to be in one place, yet be able to perceive and act like you are present in a different place. This differs from remote control by how you interface with the remote environment. With remote control, you use some type of controller and perhaps communication system to control something at a distance, but your actions and perception are that you are controlling something at a distance. For example, if you press a button that causes a light to turn on 1000 miles away, that would be remote control of that light. Telepresence, however, simulates that you are in the remote location. With this example, you would see the environment that is 1000 miles away from your actual location and you would interact with that environment as though you were actually there. Then, to turn on that light you would turn it on just as you would if you were right there. Or, if you are sitting in a flight simulator cockpit but are actually flying an airplane (that actually exists) in a remote location, and when you look out the window you see that remote

location in the same way that you would if you were actually flying the airplane that is in that location, then you are experiencing telepresence

2.3.5 Shared Control

In shared control teleoperation, the robot assists the user in accomplishing the desired task. Rather than simply executing the user's command, the robot attempts to integrate it with information from the environment, such as obstacle and/or goal locations, and it modifies its behavior accordingly. Teleoperation allows a human user to control a robotic device through a human-machine interface (HMI) (e.g., a joystick). The ability to interact with remote or inaccessible environments made this technique widespread and effective in several applications. Examples include underwater and space exploration, telepresence, and telesurgery . In direct teleoperation, the user fully controls the robot via HMI: commands are directly mapped to robot actions with no processing. However, the interface may have fewer degrees of freedom (DoFs) than the controlled robot . Moreover, user commands may be unreliable due to noise or disturbances, particularly in cognitive HMIs such as electromyographic interface or brain-computer interface (BCI) . The robot workspace may be partially visible to the user, e.g., due to camera occlusions. Finally, user skills may be compromised by motor and/or neurological disabilities. In all these cases, the difficulty of controlling the robotic device increases the user workload and generates a sense of frustration and irritation.

2.4 Master-Slave Control Algorithm

The master-slave control architecture in robotics establishes a hierarchical relationship between two systems, namely the master and the slave. This arrangement is widely employed across diverse robotic applications due to its inherent advantages in achieving precision, maintaining control, and providing flexibility. By delineating roles between the directing master and the executing slave, this architecture forms the basis for sophisticated control strategies

2.4.1 Control Strategies

In the realm of control strategies within the master-slave architecture, the emphasis is on enabling seamless interaction between the master and slave systems. These strategies are intricately designed to facilitate real-time communication and synchronization, ensuring a high degree of precision in coordinated actions. Three key control strategies in this context are position control, force control, and impedance control. Position control ensures the accurate placement of the slave in response to the master's directives, while force control empowers the master to apply forces that are faithfully transmitted to the slave. The incorporation of impedance control further enhances the adaptability of the slave, enabling it to react effectively to external forces, contributing to a more nuanced and sophisticated robotic control framework.

2.4.2 Working Agenda

Within the master-slave control paradigm, the working agenda involves a dynamic interplay between the master and slave systems. The master system issues commands, dictating the actions to be performed, and these commands are then translated and executed by the slave system. This intricate process is particularly valuable in scenarios that demand a high level of precision, such as surgical robotics, teleoperation, or the delicate handling of materials. The master device, often equipped with sensors, captures the operator's input. Subsequently, this input is processed and transmitted to the slave system, ensuring a precise replication of the master's movements or actions. This working agenda forms the backbone of applications where precise, remote, or delicate operations are essential.

2.4.3 Applications

The applications of master-slave control architectures are expansive, spanning various fields with unique requirements. In teleoperation, this architectural approach facilitates the remote control of robotic systems, empowering operators to

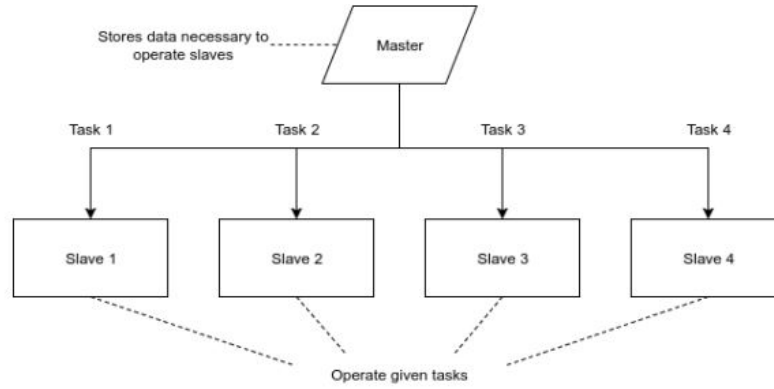


Figure 2.1: Master-Slavecontrol Architecture

manipulate robots in environments that are hazardous or challenging to access. In surgical robotics, the master-slave architecture proves invaluable for achieving precise control in minimally invasive surgeries. Here, the surgeon manipulates the master system, and the slave system replicates these movements with heightened precision, contributing to improved medical procedures. Industries requiring intricate material handling benefit significantly from the precise control offered by master-slave architectures, ensuring optimal manipulation of robotic arms or grippers in manufacturing settings. Moreover, the integration with virtual reality (VR) and augmented reality (AR) systems provides users with a means to control robotic systems for immersive experiences, training, or simulations, expanding the applicability of this architecture into novel and dynamic fields.

2.5 Communication Protocols in embedded systems

Embedded systems and microcontrollers often rely on various communication protocols to enable data exchange between different components. These protocols facilitate the seamless interaction of devices within the system. Here are some commonly used communication protocols in embedded systems and microcontrollers.

2.5.1 Inter-Integrated Circuit (I2C)

Inter-Integrated Circuit (I2C) is a popular synchronous serial communication protocol that facilitates data exchange between multiple devices connected to the same bus. Developed by Philips Semiconductor (now NXP Semiconductors), I2C has become a standard for short-range communication in embedded systems. It utilizes two lines, Serial Data (SDA) and Serial Clock (SCL), to enable communication between microcontrollers, sensors, and other peripherals. I2C is widely used in various embedded systems and microcontroller-based applications. It is particularly suitable for scenarios where multiple devices need to communicate with each other over short distances. Common applications include connecting sensors (such as temperature sensors, accelerometers, and gyroscopes), EEPROMs, real-time clocks (RTCs), and other peripherals to microcontrollers. I2C is also prevalent in applications requiring a low-cost, low-complexity communication solution.

2.5.2 Serial Peripheral Interface (SPI)

Serial Peripheral Interface (SPI) is a widely used synchronous serial communication protocol that facilitates high-speed data transfer between microcontrollers and various peripherals. Developed as a full-duplex communication standard, SPI employs multiple wires, including Master In Slave Out (MISO), Master Out Slave In (MOSI), Serial Clock (SCLK), and Chip Select (CS/SS), allowing for efficient and reliable communication.

In SPI communication, the microcontroller serves as the master device, initiating and controlling the data exchange with one or more peripheral devices acting as slaves. The Master Out Slave In (MOSI) line carries data from the master to the slaves, while the Master In Slave Out (MISO) line transmits data from the slaves back to the master. The Serial Clock (SCLK) line provides the clock signal that synchronizes data transmission. Additionally, the Chip Select (CS/SS) line is used to select the specific slave device with which the master intends to communicate.

2.5.3 Controller Area Network

Controller Area Network (CAN) is a robust and deterministic serial communication protocol that was initially developed by Bosch for use in automotive applications. Over time, its usage has expanded beyond the automotive industry to various industrial applications. CAN is designed to support communication between multiple microcontrollers and devices over a shared bus, providing a reliable and efficient means of data exchange in real-time systems.

Working Principle:

CAN operates on a multi-master, multi-slave principle, allowing multiple microcontrollers or devices to communicate on the same bus. Unlike other communication protocols, CAN does not have a fixed master-slave architecture. Instead, any node on the network can act as a sender or receiver, providing a flexible and decentralized communication structure. The protocol utilizes a non-destructive bitwise arbitration mechanism, where the node with the highest priority (lowest identifier) gains access to the bus.

CAN uses a two-wire twisted-pair configuration: the CAN High (CANH) and CAN Low (CANL) lines. This differential signaling helps in reducing electromagnetic interference and enhances the reliability of communication. The shared bus architecture enables efficient communication between electronic control units (ECUs), sensors, and actuators, forming a network within a vehicle or an industrial system.

Applications:

CAN is employed in automation and process control, connecting controllers, sensors, and other devices. Its reliability and real-time capabilities make CAN suitable for applications where high data integrity and deterministic communication are paramount.

Advantages:

- **Real-Time Determinism:** CAN ensures deterministic communication, making it suitable for applications where timely data exchange is crucial.

- **Robustness and Reliability:** The protocol's error detection mechanisms contribute to its high reliability, and its fault tolerance ensures continued operation in the presence of faults.
- **Shared Bus Architecture:** The shared bus architecture allows multiple nodes to communicate efficiently, reducing the need for complex wiring and connectors.

Challenges:

- **Limited Bandwidth:** CAN has a limited data transfer rate compared to some newer communication protocols, which may be a constraint in applications requiring higher bandwidth.
- **Limited Bandwidth:** CAN has a limited data transfer rate compared to some newer communication protocols, which may be a constraint in applications requiring higher bandwidth.
- **Message Length Limitations:** The length of CAN messages is limited, which may pose challenges in transmitting large amounts of data in a single message.

2.5.4 Bluetooth and Wi-Fi:

Bluetooth and Wi-Fi are two wireless communication protocols that have become integral components of embedded systems, providing wireless connectivity solutions for a wide range of applications. While both are used for wireless communication, they differ in their intended use cases, data rates, and power consumption.

Bluetooth:

Bluetooth is a short-range wireless communication protocol designed for scenarios where devices need to communicate over relatively short distances. It operates in the 2.4 GHz frequency band and is commonly used in applications such as wireless audio streaming, hands-free calling, and connecting peripherals like

keyboards and mice to computers or mobile devices. Bluetooth is particularly well-suited for low-power applications, making it a popular choice for wearables, health monitoring devices, and home automation systems **Wi-Fi:**

Wi-Fi, on the other hand, is designed for medium to long-range wireless communication and offers higher data rates compared to Bluetooth. It operates in the 2.4 GHz and 5 GHz frequency bands, providing more bandwidth for data-intensive applications. Wi-Fi is commonly used in home and office environments for internet connectivity, as well as in IoT (Internet of Things) applications where higher data rates are required for transferring large amounts of data

2.5.5 Modbus:

Modbus is a widely adopted serial communication protocol designed for industrial automation applications. Developed in the late 1970s, it has become a standard for communication between microcontrollers and programmable logic controllers (PLCs) in various industrial settings. Modbus provides a means for devices to exchange data over a network, enabling efficient monitoring and control within industrial automation systems.

2.5.6 Universal Asynchronous Receiver-Transmitter (UART):

The Universal Asynchronous Receiver-Transmitter (UART) stands as a fundamental communication protocol extensively used in embedded systems and microcontrollers. Serving as a reliable method for serial communication, UART facilitates the exchange of data between devices within a system. Its simplicity and versatility make it a preferred choice for a myriad of applications. UART finds applications in diverse fields due to its simplicity and reliability. It is commonly used for connecting microcontrollers with peripherals, sensors, and other microcontrollers within a system. UART is prevalent in scenarios where short-distance communication is sufficient, such as in embedded systems, IoT devices, and various consumer electronics.

Chapter 3

Preliminaries

This chapter contains preliminaries for this project.

3.1 Software

Here is the brief information about the softwares utilized in the development of system.

3.1.1 *STM32CubeMX: Configuring STM32 Microcontrollers*

STM32CubeMX, a graphical software tool developed by STMicroelectronics, streamlines the initialization process of STM32 microcontrollers. It provides an intuitive interface for configuring peripherals and generating initialization code.

Functionalities:

- **Peripheral Configuration:** Allows users to configure GPIO pins, timers, UARTs, SPI, CAN, and I2C interfaces for sensor interfaces, motor control, and component communication.
- **Clock Configuration:** Enables the setup of clock sources, dividers, and prescalers to ensure the microcontroller operates at the desired frequency.

- **Hardware Abstraction Layer (HAL) Generation:** Facilitates the creation of initialization code compatible with STM32 HAL, simplifying interaction with peripherals.
- **Middleware and Middleware Stacks:** Supports the integration of middleware components such as FreeRTOS and USB stacks into firmware.
- **Code Generation and IDE Compatibility:** Generates importable initialization code compatible with development environments like Keil μ Vision.
- **Peripherals Optimization:** Provides optimization suggestions to ensure efficient resource allocation and identifies conflicts for real-time performance.
- **Graphical Pinout and Configuration Visualization:** Offers visual representations for easy understanding and modification of microcontroller configurations.
- **Support for ST Boards and Components:** Simplifies configuration when using STM32 development boards and ST components.

STM32CubeMX ensures efficient microcontroller configurations for sensor interfacing, motor control, and communication, contributing to system efficiency and reliability.

3.1.2 Keil μ Vision

Key functionalities and advantages of Keil Vision include:

- **Specialized IDE:** Keil μ Vision is specifically tailored for ARM-based microcontrollers, notably the STM32 series, offering an environment optimized for firmware development.
- **Low-level Programming:** Adept at low-level programming, allowing developers to work intimately with the microcontroller's hardware, registers, and memory.

- **Debugging Support:** Keil μ Vision provides comprehensive debugging tools, aiding in identifying and rectifying issues within the firmware code.
- **Optimization Capabilities:** The IDE allows for code optimization, enabling developers to fine-tune the firmware for efficient and responsive control of the manipulator. This is especially important for tasks involving motion control, sensor data processing, and communication protocols.
- **Direct Hardware Interaction:** It offers the advantage of direct interaction with hardware peripherals and registers. This level of control is pivotal for customization, especially when dealing with microcontrollers integral to the project's operations.

3.1.3 *FreeRTOS*

The integration of FreeRTOS revolutionizes the master controller, fostering real-time capabilities and refined task management crucial for the serial chain manipulator's precision. Here are features of FreeRTOS embedded in the Master controller, Integration of FreeRTOS into Master Controller:

- Incorporates FreeRTOS within the master controller, effectively transforming it into a real-time operating system.
- Enables efficient management and scheduling of tasks, utilizing the benefits of mutexes and semaphores from FreeRTOS.
- Mutexes ensure exclusive access to shared resources, preventing conflicts and maintaining data integrity.
- Semaphores facilitate synchronized task execution, ensuring a coherent flow of operations.

Portability:

- FreeRTOS is designed to be architecture-agnostic, facilitating easy porting to various microcontroller architectures and platforms. This inherent portability allows developers to deploy FreeRTOS across diverse hardware configurations without significant code modifications.

Real-Time Responsiveness:

- Enhances the system's ability to respond in real-time by effectively executing tasks, including complex calculations like inverse kinematics.
- Mutexes play a critical role in preserving data integrity, preventing data corruption, and ensuring accurate critical information management.
- Semaphores control task synchronization, allowing precise coordination and avoidance of contention among concurrent tasks.

Optimized Communication:

- FreeRTOS, complemented by mutexes and semaphores, optimizes communication pathways between the master controller and other system components.
- Mutexes guarantee consistency in shared data, ensuring coherence and reliability.
- Semaphores orchestrate task synchronization, facilitating uninterrupted information flow and system operation.

Enhanced Safety and Control:

- Empowers the system to swiftly respond to safety-critical events, such as collision detection, by adjusting the manipulator's position promptly.
- Mutexes maintain reliability in safety-related data, ensuring accurate responses.

- Semaphores expedite safety actions, aiding in the timely execution of critical tasks.

Task Scheduling and Priority Management:

- Offers the capability to define task priorities and scheduling policies, ensuring crucial calculations and actions receive priority.
- Mutexes and semaphores maintain order during task execution, preventing conflicts and managing resource access efficiently.

Parallel Processing:

- Facilitates the distribution of tasks across system resources, enabling parallel processing for improved system performance.
- Mutexes maintain data integrity across multiple tasks, ensuring consistency.
- Semaphores promote synchronization among concurrent processing units, facilitating coordinated task execution.

Interrupt Handling:

- FreeRTOS offers a reliable mechanism for handling interrupts, minimizing interrupt latency. This feature is crucial for real-time embedded systems, where timely responses to external events are paramount. The RTOS efficiently manages interrupts, enabling the system to respond promptly to critical events without compromising overall performance.

The integration of FreeRTOS into the master controller forms the foundation for a sophisticated control system design. It provides real-time capabilities, responsiveness, and adaptability, ensuring the serial chain manipulator operates with unmatched precision and efficiency. Mutexes and semaphores, as integral elements of FreeRTOS, are instrumental in safeguarding data integrity, orchestrating task synchronization, and optimizing system performance.

3.1.4 *SolidWorks*

We used solid work due to the following advanced functionalities,

- **3D Modeling and Simulation:** SolidWorks is instrumental in conducting 3D modeling and simulation of the serial chain manipulator's mechanical structure.
- **Design Facilitation:** The software streamlines the design process, allowing for the visualization of the manipulator's physical components with precision.
- **Kinematic Modeling:** SolidWorks enables accurate kinematic modeling, enhancing the understanding and representation of the manipulator's motion.
- **Effective Collision Detection:** It provides robust features for collision detection, ensuring the definition of joint limits and enhancing overall safety in the manipulator's operation.
- **Critical Role in Mechanical Design and Analysis:** SolidWorks plays a pivotal role in the mechanical design and analysis of the manipulator, contributing significantly to its functionality and reliability.

3.1.5 *V-REP/coppeliasim*

V-REP (Virtual Robot Experimentation Platform) offers crucial functionalities:

- **Simulation Environment:** Provides a virtual space to replicate manipulator behavior.
- **Algorithm Testing:** Facilitates testing and refining of control algorithms.
- **Perception Module Evaluation:** Allows assessment of perception modules' performance.

- **Collision Avoidance Assessment:** Enables evaluation of collision avoidance strategies in a secure, risk-free setting.

This preliminary testing and validation process within V-REP is pivotal for refining the control system before deploying it on actual hardware.

3.2 Microcontroller

In the domain of robotic arms, a controller encompasses both hardware and software components responsible for overseeing and managing the arm's motion and functionality. It gathers input from a variety of sources, including sensors and user instructions, and employs this data to fine-tune the arm's movements and operations. For instance, a controller might employ a PID algorithm to adjust the actuators' positions, ensuring precise end effector positioning. Safety features may also be integrated within the controller to prevent collisions with objects in the arm's environment or to trigger an emergency stop. In this specific project, two types of microcontrollers were utilized to govern the three degrees of freedom exhibited by the robotic arm.

3.2.1 *Nucleo - F767ZI*

The Nucleo-F767ZI development board as shown in Figure can serve as an advanced central controller for a robotic arm, taking on a crucial role in overseeing and coordinating the arm's various functions. Equipped with a robust STM32 microcontroller featuring a Cortex-M7 core and a wide array of peripheral support, it is well-suited to manage the intricate operations of a robotic arm. In the context of being a master controller for a robotic arm, the Nucleo board fulfills several key functions:

- **Slave Motor Control:** With its processing capabilities, the STM32 microcontroller can interact with and regulate the motors and actuators of the robotic arm using slave controllers. It produces precise control signals for

slave controllers and executes motion control algorithms to ensure accurate and synchronized movements.

- **Sensor Integration:** The Nucleo board's compatibility with an assortment of sensors enables it to collect data from proximity sensors, cameras, and other sensory components embedded within the robotic arm. This data becomes invaluable for feedback control, obstacle detection, and precise positioning.
- **Real-Time Processing:** The Cortex-M7 core bestows the microcontroller with potent computational prowess and real-time processing capabilities, essential for making split-second decisions and executing intricate maneuvers with the robotic arm.
- **Communication:** The Nucleo-F767ZI offers diverse communication interfaces such as UART, CANbus, SPI, I2C, and Ethernet. These communication options enable the robotic arm to interact with other arm components, slave devices, or even a teleoperation control system.
- **Safety Features:** Safety is paramount in a robotic arm's operation. The Nucleo board can implement essential safety features, including emergency stop mechanisms, collision detection systems, and fault tolerance protocols, ensuring the secure functioning of the robotic arm.
- **Development Ecosystem:** The Nucleo platform provides an extensive suite of development tools and software libraries. These resources facilitate the rapid prototyping, programming, and testing of the robotic arm's control algorithms and logic.
- **Floating Point Calculations:** The board's capability to perform floating-point calculations is a critical asset for complex computations, aiding in the robotic arm's ability to execute intricate and precise movements with utmost accuracy.



Figure 3.1: STM32 Nucleo-144 development board

- **Inverse Kinematics:** Inverse kinematics calculations are paramount for translating desired end-effector positions into joint angles, ensuring that the robotic arm can achieve specific tasks with precision. The computational prowess of the Nucleo board allows it to handle these calculations efficiently, enabling the arm to reach desired positions and orientations in its workspace.

In summary, the Nucleo-F767ZI development board stands as an adept master controller for a robotic arm, equipped with processing power, sensor integration, versatile communication capabilities, safety protocols, and expansive development tools. Furthermore, it excels in handling floating-point calculations and inverse kinematics, critical for executing intricate and precise robotic arm movements. This collective set of capabilities ensures that the board effectively governs the operation of robotic arms, promoting efficiency, safety, and precision in a variety of applications.

3.2.2 STM32F103C8T6 Blue Pill

The STM32F103C8T6 Blue Pill development board is serving as a slave controller. It is ideally suited for applications where precise control over the position of a

motor is critical. This is the way that motor position control is achieved using the Blue Pill board.

- **Microcontroller Capabilities:** The STM32F103C8T6 microcontroller, with its ARM Cortex-M3 core and clock speed of up to 72 MHz, is well-equipped to handle motor position control tasks. It can perform the necessary calculations for real-time position control.
- **Motor Control Peripherals:** The microcontroller offers a wide range of peripherals, including advanced timers, PWM channels, and encoder interfaces. These features are essential for generating precise control signals, reading motor position feedback from encoders, and adjusting the motor's position as needed.
- **Real-Time Processing:** Motor position control requires rapid and accurate adjustments based on encoder feedback. The Blue Pill's real-time processing capabilities enable it to execute these adjustments swiftly and with precision.
- **GPIO Flexibility:** The GPIO pins on the board can be configured to interface with motor drivers, encoders, and limit switches. This flexibility allows the Blue Pill to manage the communication between the motor, feedback sensors, and other control components.
- **Communication Interfaces:** The STM32 microcontroller on the Blue Pill provides communication interfaces such as UART, CANbus, SPI, and I2C, which can be employed for external communication and data exchange with other control units or central/master controllers.

In conclusion, the STM32F103C8T6 Blue Pill development board, as shown in Figure [3.2], serves as an effective slave controller. It receives input from encoders, processes the data using the PID algorithm, and subsequently regulates the motor's position in real-time to ensure it aligns with the desired position accurately.



Figure 3.2: STM32 Bluepill development board

and with precision. The board's capabilities, combined with its real-time processing and feedback integration features, make it a valuable solution for applications demanding precise motor position control.

3.3 Communications

Following are the major communication protocols that will be used in this thesis,

3.3.1 *CAN Bus - Controller Area Network*

Robotic arms, composed of multiple embedded controllers, rely on efficient information exchange for precise coordination and seamless operation. The Controller Area Network (CAN) protocol serves as a robust communication backbone within robotic arms, ensuring reliability and real-time performance. **The CAN Protocol:** In the complex interplay of various robotic arm components requiring precise control and synchronization, the CAN protocol, functioning as a broadcast-type communication system, seamlessly integrates. All controllers within the arm receive all transmissions, eliminating the need for intricate addressing schemes.

Message Types: CAN defines four primary message types, each with specific communication roles:

- **Data Frame:** The backbone of CAN communication, encompassing an Arbitration Field for message priority, a Data Field crucial for motor positions or sensor data, a CRC Field for error detection, and an Acknowledgment Slot for confirmation of successful reception.
- **Remote Frame:** Utilized when one controller needs specific data from another, serving as a polite request for particular information.
- **Error Frame:** Sent upon fault detection, triggering a robust error confinement algorithm to prevent one node from disrupting the entire system.
- **Overload Frame:** Issued when a controller becomes too busy to respond immediately, signaling others to temporarily wait for synchronization until it catches up.

Standard vs. Extended CAN: The diversity of controllers within a robotic arm necessitates Extended CAN (CAN 2.0B) with its 29-bit identifiers. This capability allows a wide range of unique identifiers, crucial for prioritizing and distinguishing between various controllers for efficient operation.

Bus Arbitration and Message Priority: Precise coordination of robotic arm movements hinges on timely commands. CAN's arbitration mechanism grants priority to the controller with the lowest numerical identifier in cases of simultaneous transmission attempts. This minimizes conflicts and ensures precise real-time control.

Seamless Communication Within the System: CAN messages are "content-addressed," where their contents implicitly determine their address. This approach simplifies communication between controllers responsible for different aspects of the robotic arm's operation.

In robotics, efficient communication is paramount for precision, reliability, and real-time control. The CAN protocol, with its diverse message types, arbitration mechanisms, and flexibility, provides seamless communication among embedded



Figure 3.3: MCP2551 CAN module for communication between master and slaves controllers. It ensures harmonious operation, facilitating precise movements and exceptional performance. As robotic arm applications expand across industries.

3.4 Limit switches

U-shaped sensor(LM393) can be use as limit switch.

3.4.1 *Sensor Module Specifications:*

- **Chip:** LM393, Groove Coupler H2010

The sensor module utilizes the LM393 comparator chip for efficient signal processing.

- **Groove Coupler Width:** 10 mm

The groove coupler has a width of 10 mm, providing precise detection capabilities for obstacles passing through the slot.

- **Working Voltage:** 5 VDC

The sensor module operates at a low voltage of 5 volts, making it compatible with common electronic systems and microcontrollers.

- **Size:** 27.6 mm x 20 mm x 17 mm

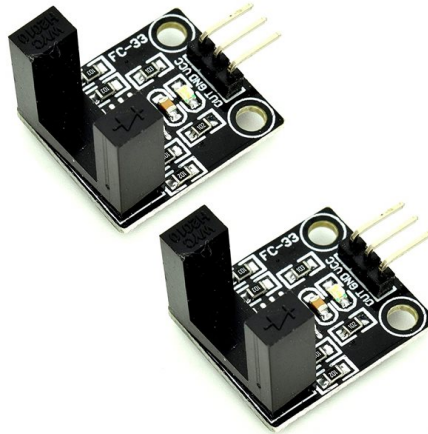


Figure 3.4: U-shaped sensor module used as a limit switch

The compact dimensions of the sensor module (27.6 mm x 20 mm x 17 mm) facilitate easy integration into robotic systems or other projects with space constraints.

3.4.2 *Limit Switch Functionality:*

U-shaped sensor module is implemented as a limit switch to ensure the safety and controlled operation of the robotic arm. The limit switch functionality is crucial for preventing unintended movements and protecting both the robotic system and its surroundings. Here's how the limit switch functionality works:

- **Mounting and Alignment:** The U-shaped sensor module is strategically mounted in such a way that the U-shaped groove coupler is aligned with the range of motion of the robotic joints. This alignment ensures that the sensor can effectively detect any obstacles or surfaces passing through the groove.
- **Specified Limits Configuration:** The project's control system is configured to recognize specific limits within the range of motion for each robotic joint. These limits are predetermined based on the mechanical design and requirements of the robotic arm.
- **Continuous Monitoring:** As the robotic arm moves, the U-shaped sensor module continuously monitors the space within its groove coupler. The

LM393 comparator chip processes the input signals, maintaining awareness of any obstructions or changes in the detected environment.

- **Triggering a Digital Signal:** When a robotic joint approaches or reaches its specified limit, the U-shaped sensor module triggers a clean and stable digital signal. This signal is a reliable indication that the limit has been reached.
- **Motor Shutdown Signal:** The digital signal generated by the sensor module is then used as a trigger to initiate a motor shutdown sequence. This sequence ensures that power to the motor driving the respective robotic joint is cut off immediately.
- **Preventing Unintended Movements:** By shutting down the motor upon reaching the specified limits, the U-shaped sensor module prevents unintended movements of the robotic arm beyond its safe operational range. This enhances the overall safety and control of the robotic system.
- **Safety and System Integrity:** The integration of the U-shaped sensor as a limit switch contributes to the safety and integrity of the entire robotic control system. It acts as a fail-safe mechanism, reducing the risk of collisions, damage, or unsafe operation.

In summary, the U-shaped sensor module, configured as a limit switch, plays a pivotal role in monitoring and controlling the robotic arm's movements. By generating a digital signal upon reaching specified limits, it facilitates the immediate shutdown of the motor, ensuring safe and controlled operation of the robotic system.

Chapter 4

Kinematic Modeling

4.1 CAD Model of the robot

This is the cad model of our extended degree of freedom robot.

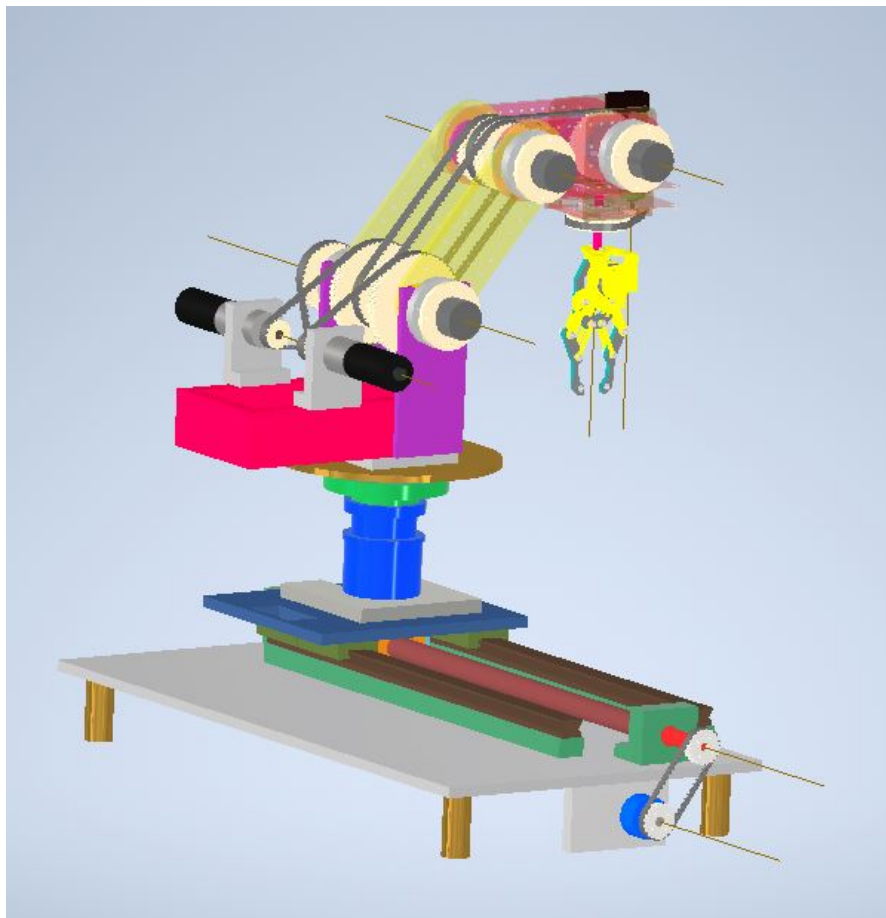


Figure 4.1: Cad model of the extended -DOF robot

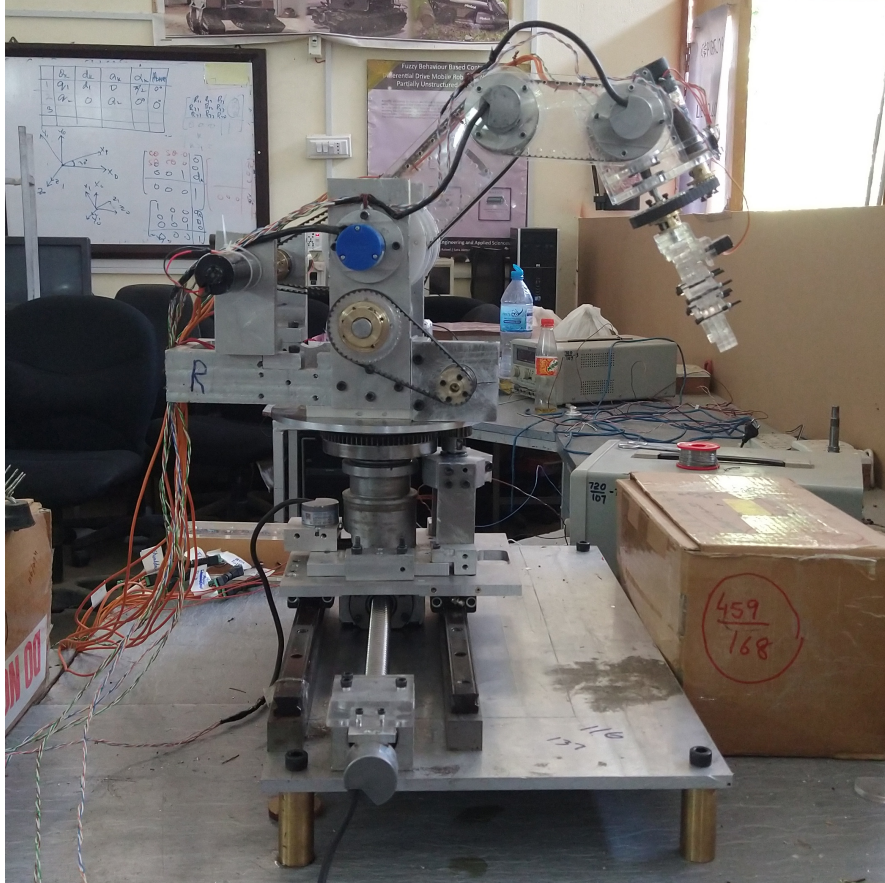


Figure 4.2: Extended 5 DOF robot

4.2 Forward Kinematics

Forward kinematics uses a robot's kinematic equations to calculate the end-effector's position from specified joint parameter values. The Denavit-Hartenberg (DH) parameters used to describe the forward kinematics of robotic manipulators.

4.2.1 Denavit-Hartenberg Parameters

For the DH parameters first we draw a Link co-ordinate diagram of the robot. DH parameter includes (α, a, d, θ) for each joint in extended 5-axis robot.

α : Link twist angle.

a : Link length.

d : Link offset.

θ : Joint angle.

General transformation matrix is derived below which will be used for deriving

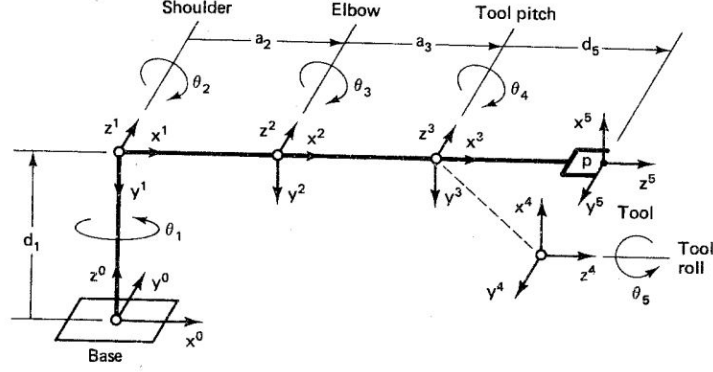


Figure 4.3: Link Co-ordinate diagram

homogeneous transformation matrix for each joint using the DH parameters. The steps involved in the derivation of the transformation matrix are as follows:

$$\mathbf{A}_i = \mathbf{Rot}_{z,d_i} \mathbf{Tran}_{z,d_i} \mathbf{Tran}_{x,\alpha_i}$$

where:

$\mathbf{Rot}_{z,d_i}$ represents a rotation matrix about the z -axis by d_i radians,

$\mathbf{Tran}_{z,d_i}$ represents a translation matrix along the z -axis by a distance d_i ,

$\mathbf{Tran}_{x,\alpha_i}$ represents a translation matrix along the x -axis by a distance α_i .

$$\mathbf{A}_i = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) & 0 & 0 \\ \sin(\theta_i) & \cos(\theta_i) & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & \alpha_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{A}_i = \begin{bmatrix} \cos(\theta_i) & \sin(\theta_i) & 0 & 0 \\ -\sin(\theta_i) \cos(\alpha_i) & \cos(\theta_i) \cos(\alpha_i) & \sin(\alpha_i) & 0 \\ \sin(\theta_i) \sin(\alpha_i) & -\cos(\theta_i) \sin(\alpha_i) & \cos(\alpha_i) & 0 \\ a_i \cos(\theta_i) & a_i \sin(\theta_i) & d_i & 1 \end{bmatrix}$$

Let a_i denote the distance between Z_{i-1} and Z_i measured along X_i . Let α_i denote the angle from Z_{i-1} to Z_i measured about X_i . Let d_i represent the distance

between O_{i-1} to the intersection of Z_{i-1} and Z_i measured along Z_{i-1} . The angle from X_{i-1} to Z_i measured about Z_{i-1} is given by θ_i from the link coordinate diagram of the robot given below,

DH parameter table:

Link	θ_i	α_i	a_i	d_i
1	θ_1	-90	0	296.65
2	θ_2	0	162.78	0
3	θ_3	0	99.86	0
4	θ_4	-90	0	0
5	θ_5	0	0	268.43

$$T = \begin{bmatrix} c_1 & 0 & -s_1 & 0 \\ s_1 & 0 & c_1 & 0 \\ 0 & -1 & 0 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} c_2 & -s_2 & 0 & a_2 c_2 \\ s_2 & c_2 & 0 & a_2 s_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} c_3 & -s_3 & 0 & a_3 c_3 \\ s_3 & c_3 & 0 & a_3 s_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{\text{base}}^{\text{wrist}} = \begin{bmatrix} c_1 c_{23} & -c_1 s_{23} & -s_1 & c_1(a_2 c_2 + a_3 c_2 c_3) \\ s_1 c_{23} & -s_1 s_{23} & c_1 & s_1(a_2 c_2 + a_3 c_2 c_3) \\ s_{23} & -c_{23} & 0 & d_1 - a_2 s_2 - a_3 s_{23} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T = \begin{bmatrix} c_4 & 0 & -s_4 & 0 \\ s_4 & 0 & c_4 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} c_5 & -s_5 & 0 & 0 \\ s_5 & c_5 & 0 & 0 \\ 0 & 0 & 1 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{\text{wrist}}^{\text{tool}} = \begin{bmatrix} c_4 c_5 & -c_4 s_5 & -s_4 & -d_5 s_4 \\ s_4 c_5 & -s_4 s_5 & c_4 & d_5 c_4 \\ -s_5 & -c_5 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Final matrix from base to tool is,

$$T_{\text{base to tool}} = \begin{bmatrix} c_1 c_{234} c_5 + s_1 s_5 & -c_1 c_{234} s_5 + s_1 c_5 & -c_1 s_{234} & c_1 (a_2 c_2 + a_3 c_{23} - d_5 s_{234}) \\ s_1 c_{234} c_5 + c_1 s_5 & -s_1 c_{234} s_5 - c_1 c_5 & s_1 s_{234} & s_1 (a_2 c_2 + a_3 c_{23} - d_5 s_{234}) \\ -s_{234} c_5 & s_{234} s_5 & -c_{234} & d_1 - a_2 s_2 + a_3 s_{23} - d_5 c_{234} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.3 Inverse kinematics

Inverse kinematic problem starts with the expression for the tool configuration vector $\mathbf{w}(\mathbf{q})$, which can be obtained from the arm matrix developed in the forward kinematics solution as follows:

$$\mathbf{w}(\mathbf{q}) = \begin{bmatrix} c_1 (a_2 c_2 + a_3 c_{23} - d_5 s_{234}) \\ s_1 (a_2 c_2 + a_3 c_{23} - d_5 s_{234}) \\ d_1 - a_2 s_2 - a_3 s_{23} - d_5 c_{234} \\ -\exp\left(\frac{q_5}{\pi}\right) c_1 s_{234} \\ -\exp\left(\frac{q_5}{\pi}\right) s_1 s_{234} \\ -\exp\left(\frac{q_5}{\pi}\right) c_{234} \end{bmatrix}$$

where $\mathbf{w}(\mathbf{q}) = [w_1, w_2, w_3, w_4, w_5, w_6]$.

Since this is an articulated robot, $q = \theta$. The notation c_{ijk} is shorthand for $\cos(\theta_i + \theta_j + \theta_k)$.

Base Joint

$$\begin{bmatrix} \frac{w_2}{w_1} \end{bmatrix} = \begin{bmatrix} \frac{s_1 (a_2 c_2 + a_3 c_{23} - d_5 s_{234})}{c_1 (a_2 c_2 + a_3 c_{23} - d_5 s_{234})} \end{bmatrix} \Rightarrow \tan(q_1) = \frac{w_2}{w_1}$$

Elbow Joint

$$\begin{bmatrix} \frac{-(c_1 w_4 + s_1 w_5)}{w_6} \end{bmatrix} = \begin{bmatrix} \frac{s_{234}}{c_{234}} \end{bmatrix} \Rightarrow q_{234} = \tan^{-1} \left(\frac{-c_1 w_4 + s_1 w_5}{-w_6} \right)$$

with intermediate variables:

$$b_1 = c_1 w_1 + s_1 w_2 + d_5 s_{234}$$

$$b_2 = d_1 - d_5 c_{234} - w_3$$

Substituting the expressions for the components of $\mathbf{w}$ into the above equations, we have:

$$b_1 = a_2 c_2 + a_3 c_{23} \quad \text{and} \quad b_2 = a_2 s_2 + a_3 s_{23}$$

The elbow angle can be isolated by computing $\|\mathbf{b}\|$, and using trigonometric identities:

$$\begin{aligned} \|\mathbf{b}\| &= \sqrt{a_2^2 + 2a_2 a_3 c_3 + a_3^2} \\ q_3 &= \pm \cos^{-1} \left(\frac{\|\mathbf{b}\|^2 - a_2^2 - a_3^2}{2a_2 a_3} \right) \end{aligned}$$

Shoulder Joint

$$\begin{aligned} \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} &= \begin{bmatrix} (a_2 + a_3 c_3)c_2 - (a_2 s_3)s_2 \\ (a_2 + a_3 c_3)s_2 + (a_3 s_3)c_2 \end{bmatrix} \\ c_2 &= \frac{(a_2 + a_3 c_3)b_1 + a_3 s_3 b_2}{\|\mathbf{b}\|^2} \\ s_2 &= \frac{(a_2 + a_3 c_3)b_2 - a_3 s_3 b_1}{\|\mathbf{b}\|^2} \\ q_2 &= \tan^{-1} \left(\frac{(a_2 + a_3 c_3)b_2 - a_3 s_3 b_1}{(a_2 + a_3 c_3)b_1 + a_3 s_3 b_2} \right) \end{aligned}$$

Tool Pitch

$$q_4 = q_{234} - q_2 - q_3$$

Tool Roll

$$\begin{aligned} q_5 &= \pi \ln \left(\sqrt{w_4^2 + w_5^2 + w_6^2} \right) \\ q_5 &= \text{atan2}(s_1 R_{11} - c_1 R_{21}, s_1 R_{12} - c_1 R_{22}) \end{aligned}$$

Chapter 5

Simulations

5.1 Forward kinematics

The equation were implemented on MATLAB and for given angles, it was verified that the forward kinematics results obtained from MATLAB ,gave the same values in the simulation and the from the values obtained from those specific angles, it was checked that the inverse was correct or not and from the simulation and MATLAB it was verified that the inverse kinematics of the robot was correct. Forward

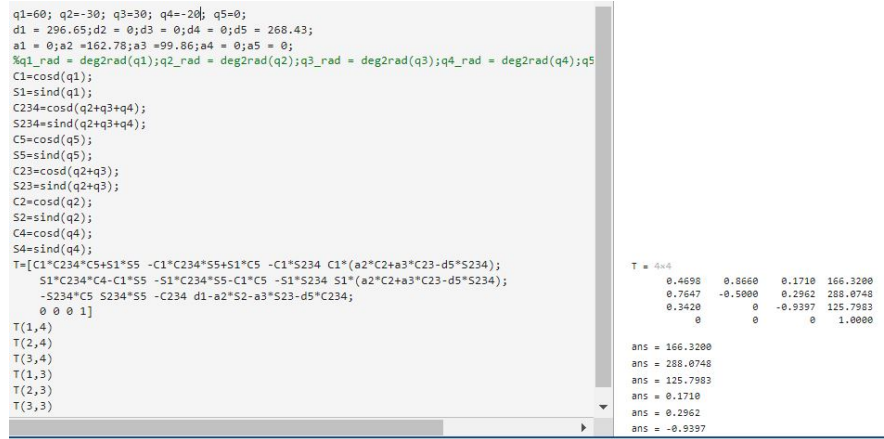


Figure 5.1: Forward Kinemetics

kinematics involves calculating the end-effector's position and orientation based on the joint angles or lengths. we verified that the results obtained from MATLAB matched the simulation, indicating that the forward kinematics implementation is accurate. we performed a simulation in V-REP, a robot simulation environment.

During the simulation, we observed that the tool tip of the robot moved to the exact coordinates that were calculated in MATLAB. This alignment between the simulated and calculated coordinates further confirms the accuracy of our forward and inverse kinematics implementations. The verification process involved comparing results obtained from MATLAB with those obtained from a simulation in V-REP. The consistency between the two provides confidence in the correctness of our kinematics implementations for the robotic system

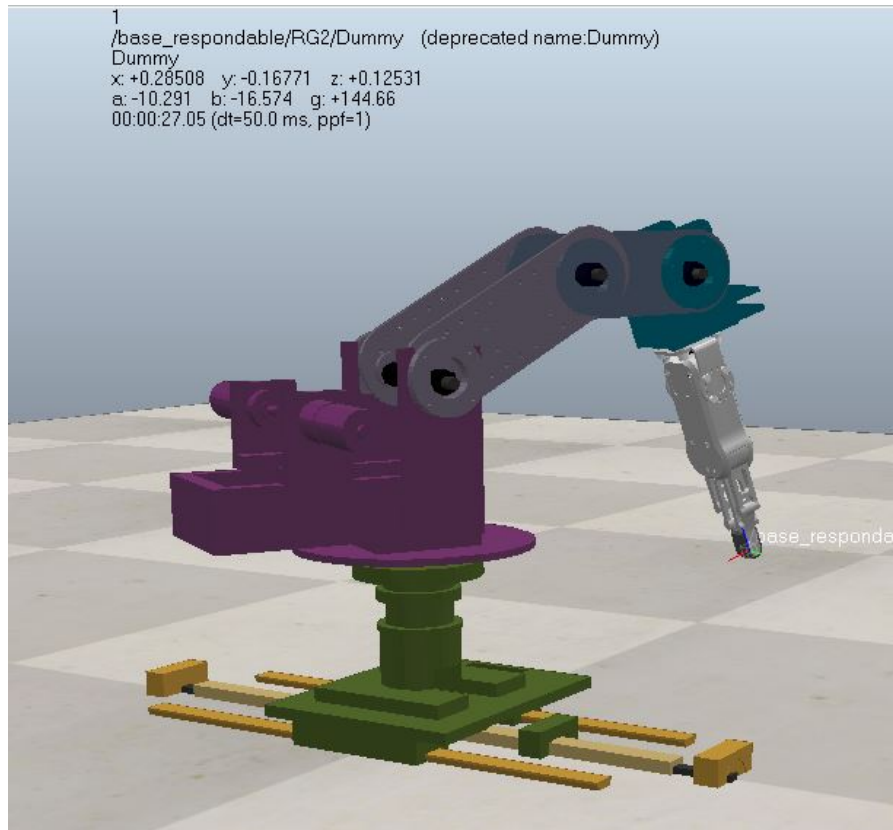
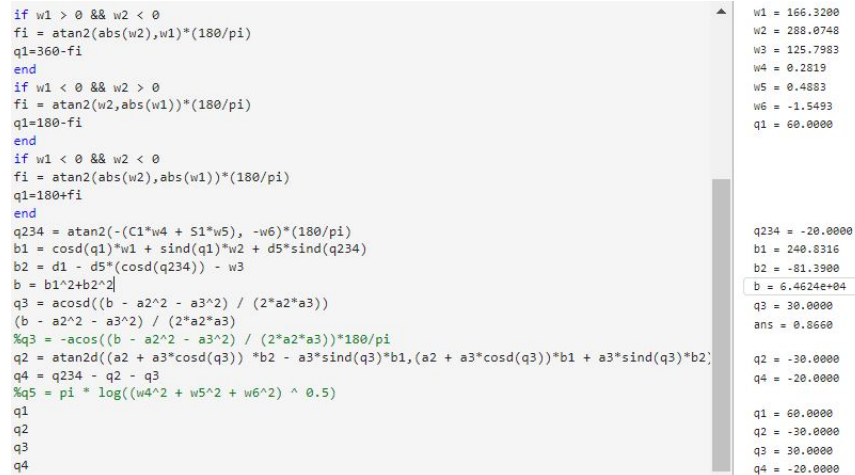


Figure 5.2: Verification of forward kinematics from simulation

5.2 Inverse Kinametics

For given joint angles, we used the forward kinematics results from MATLAB to check the correctness of inverse kinematics. Inverse kinematics involves finding the joint angles or lengths required to achieve a specific end-effector pose. It was confirmed that the inverse kinematics results obtained from MATLAB were consistent with the simulation, verifying the accuracy of the inverse kinematics

implementation. For further verification we checked manually by given angles to



```

if w1 > 0 && w2 < 0
    fi = atan2(abs(w2),w1)*(180/pi)
    q1=360-fi
end
if w1 < 0 && w2 > 0
    fi = atan2(w2,abs(w1))*(180/pi)
    q1=180-fi
end
if w1 < 0 && w2 < 0
    fi = atan2(abs(w2),abs(w1))*(180/pi)
    q1=180+fi
end
q234 = atan2(-(C1*w4 + S1*w5), -w6)*(180/pi)
b1 = cosd(q1)*w1 + sind(q1)*w2 + d5*sind(q234)
b2 = d1 - d5*(cosd(q234)) - w3
b = b1^2+b2^2
q3 = acosd((b - a2^2 - a3^2) / (2*a2*a3))
(b - a2^2 - a3^2) / (2*a2*a3)
%q3 = -acosd((b - a2^2 - a3^2) / (2*a2*a3))*180/pi
q2 = atan2d((a2 + a3*cosd(q3)) *b2 - a3*sind(q3)*b1,(a2 + a3*cosd(q3))*b1 + a3*sind(q3)*b2)
q4 = q234 - q2 - q3
%q5 = pi * log((w4^2 + w5^2 + w6^2) ^ 0.5)
q1
q2
q3
q4

```

```

w1 = 166.3280
w2 = 288.0748
w3 = 125.7983
w4 = 0.2819
w5 = 0.4883
w6 = -1.5493
q1 = 60.0000

q234 = -20.0000
b1 = 240.8316
b2 = -81.3900
b = 6.4624e+04
q3 = 30.0000
ans = 0.8660

q2 = -30.0000
q4 = -20.0000

q1 = 60.0000
q2 = -30.0000
q3 = 30.0000
q4 = -20.0000

```

Figure 5.3: Verification of Inverse kinematics from simulation

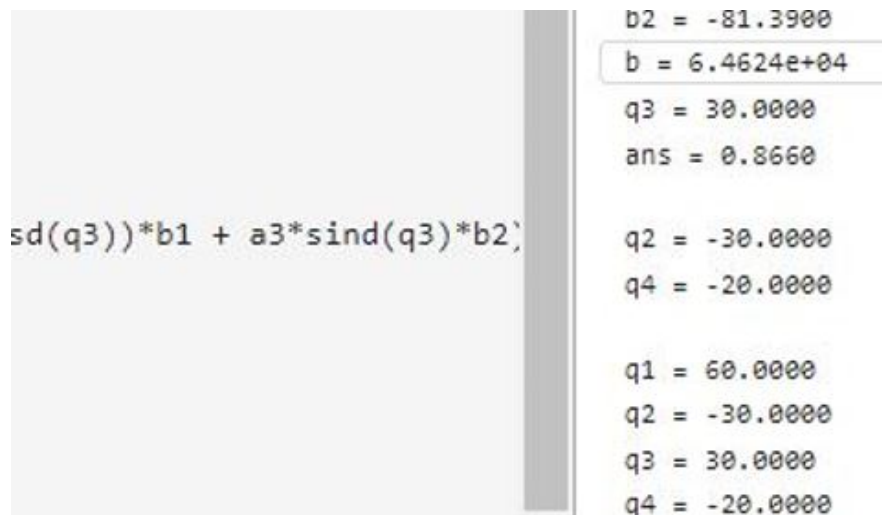
the robot in the code and checked the tooltip position then we verified that by giving those angles in MATLAB code, a saw that the inverse kinematics gave us the same angles which lead us to the same position of the tooltip.

```

local targetPos1 = {0, 60 * math.pi / 180, -30 * math.pi / 180,
30 * math.pi / 180, -20 * math.pi / 180, 0 * math.pi / 180}
moveToConfig(jointHandles, maxVel, maxAccel, maxJerk, targetPos1)

```

Figure 5.4: giving angles to the robot



```

sd(q3))*b1 + a3*sind(q3)*b2,

```

```

b2 = -81.3900
b = 6.4624e+04
q3 = 30.0000
ans = 0.8660

q2 = -30.0000
q4 = -20.0000

q1 = 60.0000
q2 = -30.0000
q3 = 30.0000
q4 = -20.0000

```

Figure 5.5: giving angles to the robot

5.3 Work Space

The worksapce of the Extended-5Dof Robot is shown in the figures

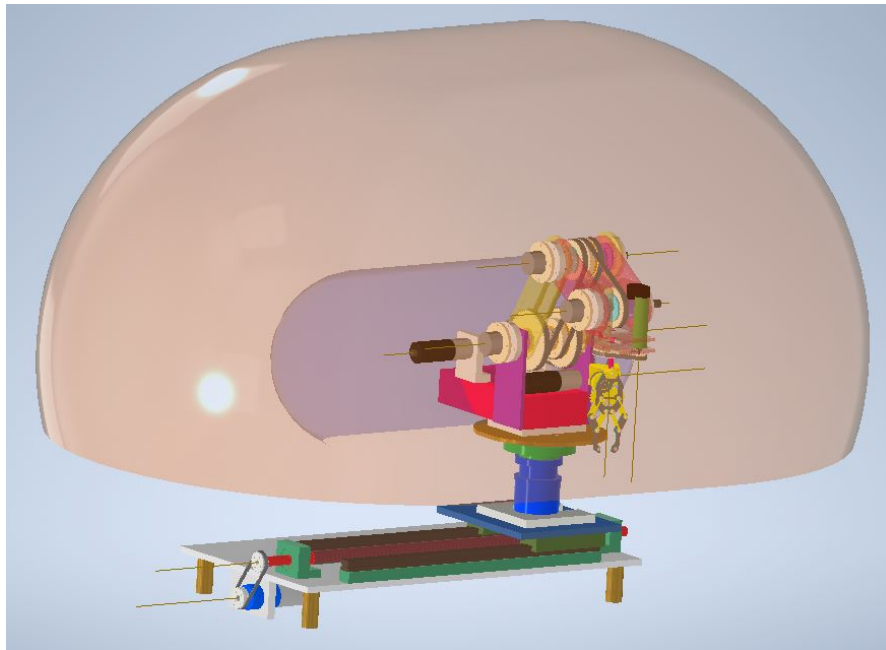


Figure 5.6: Wrokspace envelope of the robot

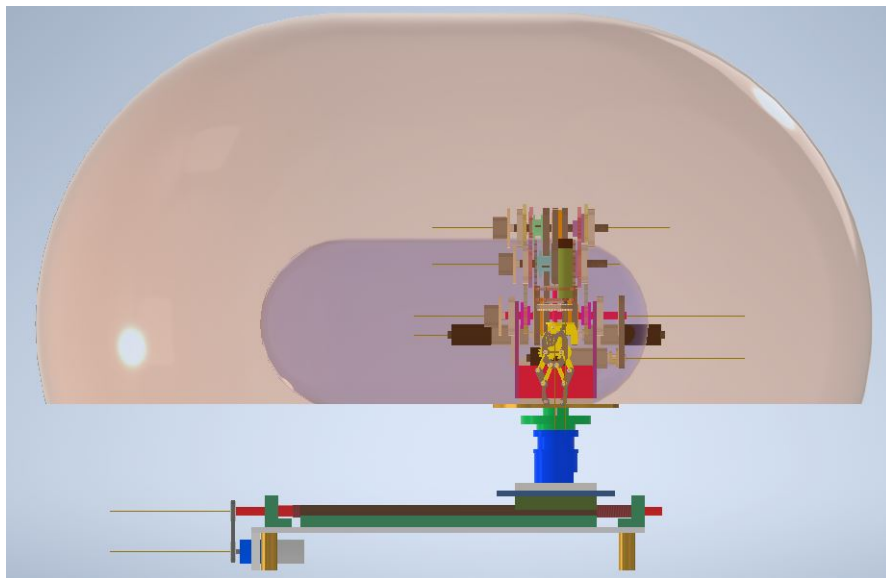


Figure 5.7: Front view of the workspace

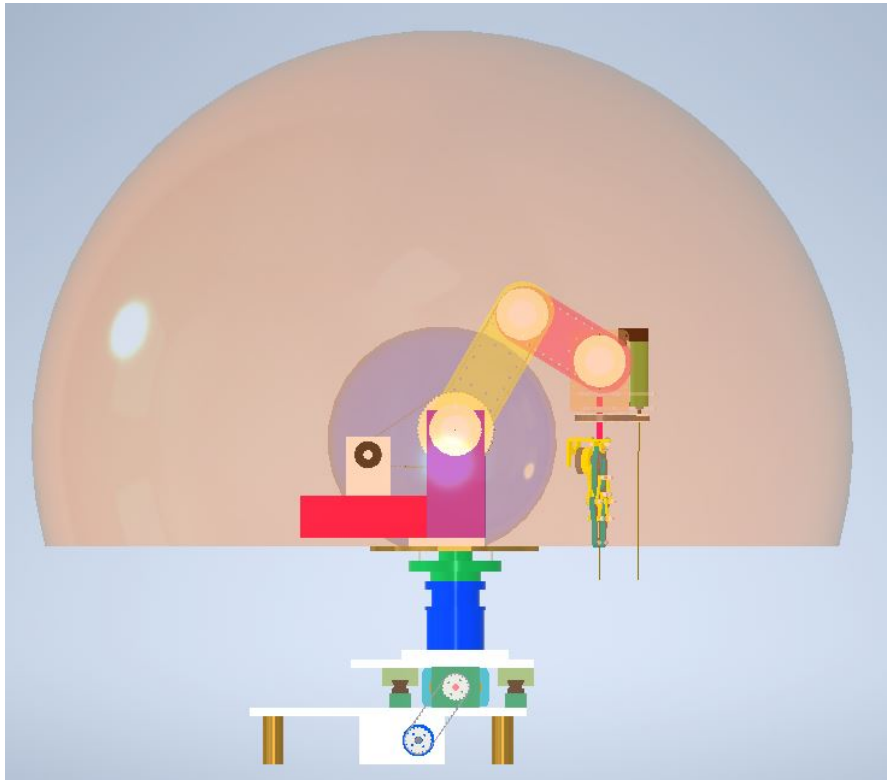


Figure 5.8: Side view of the workspace

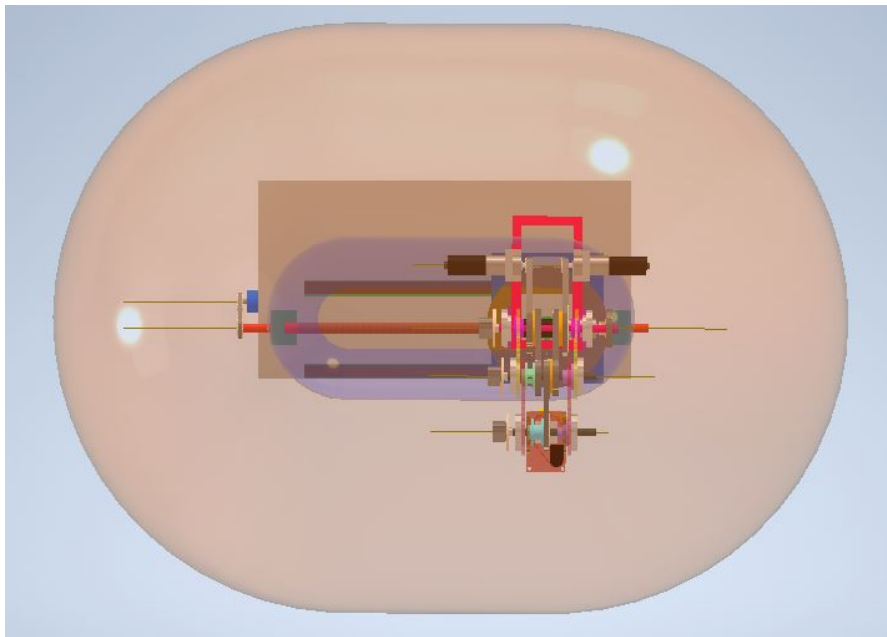


Figure 5.9: Top view of the workspace

Chapter 6

Motor control

Efficient motor control is very important aspect of robotic arm control operation, serving as the backbone for converting the master controller's commands into desirable movements. It ensures that the robotic arm's DC motors execute commands accurately and reliably, facilitating smooth and effective system performance. This precision is crucial in various applications, from manufacturing to healthcare, where finely controlled robotic arm movements are essential for optimal results. Incorporating motor control into the control algorithm of a robotic arm involves several key considerations and techniques to ensure precise and safe operation

6.1 DC Motor Analysis

DC motor analysis is very important for the design and development of control system of robotic arm. DC motor modeling and analysis is the foundation of motor control. It requires a detailed understanding of the DC motors used in the robotic arm, including their torque-speed curves, electrical and mechanical dynamics, and control interfaces. Accurate motor models are essential for designing control strategies that effectively drive the motors to achieve the desired motions. These models enable the control algorithm to predict and respond to motor behavior accurately.

6.2 Position and Velocity Control

The control algorithm encompasses techniques for managing two fundamental motor aspects: position and velocity. Position control ensures the precise positioning of the robotic arm by commanding the motor to reach specific joint angles or end effector positions. Velocity control regulates the speed of the DC motors, typically achieved through voltage control using motor drivers.

6.3 Real-Time Sensor Feedback

Motor control heavily relies on sensor feedback to continuously monitor and adjust the motor's behavior. Various types of sensors, such as position encoders and velocity sensors, provide critical feedback on the actual state of the motors. This feedback is very important in close loop control. Sensor readings are used to compare the desired state, as commanded by the operator, with the actual state. Based on this comparison, the control algorithm generates control signals that drive the motor toward the desired state. Sensor fusion techniques, which combine inputs from multiple sensors, can be employed to further improve the accuracy of motor control.

6.4 Feedback-Control Mechanisms

Closed-loop control is a fundamental concept in design and development of motor control algorithms. It ensures that the motor's behavior follows the desired state. In this mechanism, the control algorithm continuously monitors feedback from the sensors (i.e. counts from encoder) and makes real-time adjustments to the control signals sent to the motor drivers. Closed-loop control is crucial for accurate trajectory tracking, system stability, and robust responses to environmental changes or dynamic alterations.

6.5 Motor Control Techniques

Motor control algorithm incorporates a variety of motor control techniques, depending on the specific application requirements. These techniques includes:

6.5.1 *Trapezoidal Velocity Control:*

Trapezoidal velocity control is utilized to manage the velocity profiles of robotic arm motors. It involves structured acceleration and deceleration phases to ensure smooth motion. Here's how it works:

1. **Acceleration Phase:** The motor accelerates gradually from the initial velocity to the desired velocity. This phase allows the system to avoid sudden changes in velocity, which could lead to instability or mechanical stress.

$$a(t) = \frac{V_{\max}}{t_{\text{acc}}}$$

where $a(t)$ is the acceleration, $V_{\max}$ is the maximum velocity, and t_{acc} is the acceleration time.

2. **Constant Velocity Phase:** Once the desired velocity is reached, the motor maintains a constant velocity. This phase ensures smooth and consistent motion during the active movement of the robotic arm.

$$v(t) = V_{\max},$$

where $v(t)$ is the velocity and $V_{\max}$ is the maximum velocity.

3. **Deceleration Phase:** As the arm approaches its target position, the motor decelerates smoothly to come to a standstill. This phase prevents abrupt stops, reducing vibrations and ensuring precise positioning.

$$a(t) = -\frac{V_{\max}}{t_{\text{dec}}}$$

where $a(t)$ is the deceleration, t_{dec} is the deceleration time, and V_{max} is the maximum velocity.

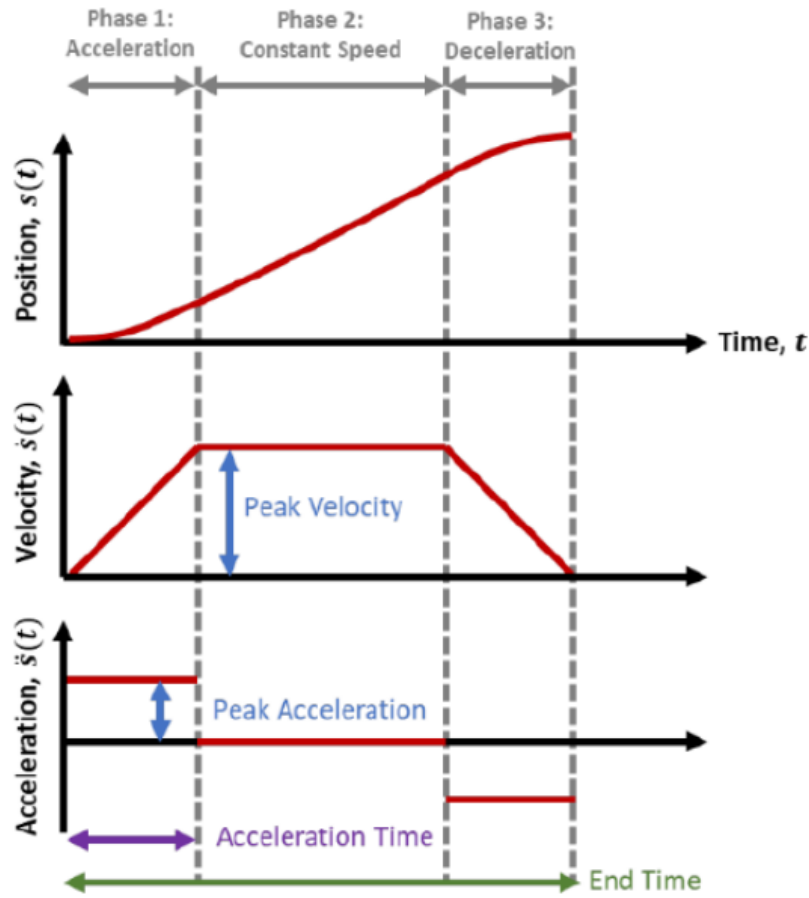


Figure 6.1: Position, Velocity, and Accelerations in a trapezoidal velocity motion profile

6.5.2 Cubic Polynomial with Time Slicing:

This technique offers precise and smooth motion control by defining motion profiles using cubic polynomials. Time slicing divides the motion into smaller segments, allowing for incremental adjustments. Here's how it operates:

1. **Motion Profile Generation:** Cubic polynomials are used to define smooth motion profiles, including position, velocity, and acceleration profiles. These profiles ensure seamless transitions between different motion states.

2. **Time Slicing:** The motion is divided into smaller time segments, known as time slices. Within each time slice, the control algorithm calculates the desired motor position or velocity based on the predefined motion profile.
3. **Incremental Adjustments:** Time slicing enables the control algorithm to make incremental adjustments to the motor's position or velocity in real-time. This ensures accurate path tracking and precise control over the robotic arm's movements.

The cubic polynomial equation for position $x(t)$ can be expressed as:

$$x(t) = a_0 + a_1t + a_2t^2 + a_3t^3$$

where a_0 , a_1 , a_2 , and a_3 are coefficients calculated based on initial and final positions, velocities, and accelerations.

The time slicing approach divides the motion into smaller time intervals Δt , allowing for incremental adjustments:

$$\Delta t = \frac{\text{total time}}{\text{number of slices}}$$

6.5.3 Proportional-Integral-Derivative (PID) Control:

PID control is a widely adopted method for regulating motors in robotic arms. It continuously assesses the error between the desired and actual motor states and computes control signals using proportional, integral, and derivative terms. Here's how it functions:

1. **Error Calculation:** PID control calculates the error, which is the difference between the desired motor state (position, velocity, or torque) and the actual motor state obtained from sensor feedback.
2. **Proportional Term:** The proportional term amplifies the error signal, providing a response proportional to the current error. This term ensures

that the motor responds quickly to changes in the desired state.

$$P(t) = K_p \cdot e(t)$$

where K_p is the proportional gain and $e(t)$ is the error.

3. **Integral Term:** The integral term accumulates past errors over time and addresses any steady-state error. It helps the system reach the desired state more accurately and eliminates long-term deviations from the target.

$$I(t) = K_i \int_0^t e(\tau) d\tau$$

where K_i is the integral gain and $e(\tau)$ is the error over time.

4. **Derivative Term:** The derivative term anticipates future errors by considering the rate of change of the error signal. It helps to dampen oscillations and improve system stability, especially during rapid changes in the desired state.

$$D(t) = K_d \frac{d}{dt} e(t)$$

where K_d is the derivative gain and $\frac{d}{dt}e(t)$ is the rate of change of the error.

The control signal $u(t)$ is the sum of these three terms:

$$u(t) = P(t) + I(t) + D(t)$$

By adjusting the gains K_p , K_i , and K_d , the PID controller can achieve the desired control performance, such as response speed, stability, and robustness.

Chapter 7

Hardware implementation

Within the field of robotics, a 6-DOF serial chain manipulator is an advanced and flexible robotic arm that resembles the complex motions of an actual human arm. Six "degrees of freedom" are regarded as conventional for most industrial and scientific applications; each "degree of freedom" refers to a joint or axis for movement. This configuration makes the manipulator a vital instrument for research, production, automation, and medical procedures since it allows the manipulator to position and orient its end effector (tool or gripper) in any direction in space.

7.1 Hardware Components

Following are the components used in implementing the hardware of 6 DOF serial Chain Manipulator:

1. **STM32F103C8T6:** These microprocessors serve as the core controllers for the manipulator, with one acting as the master to compute inverse kinematics and send commands, and six others functioning as slaves to control each joint based on received instructions.
2. **DC Motors:** Each joint of the manipulator is actuated by a DC motor, which provides the necessary rotational force to achieve precise joint movements as dictated by the slave controllers.

3. **CANbus Modules(MCP2551)**: These modules facilitate robust and reliable communication between the master and slave controllers via the CAN bus protocol, ensuring synchronized control across all joints.
4. **12V Power Supply**: This power source provides the necessary voltage to drive the DC motors and supply power to the microcontrollers and other electronic components of the manipulator.
5. **Encoders**: Attached to each joint, encoders provide feedback on the joint's position by measuring the rotation and converting it into encoder counts, which are essential for precise control and positioning.
6. **Motor Drivers(IBT-2)**: These drivers interface between the slave controllers and the DC motors, amplifying the control signals to the appropriate power levels needed to drive the motors effectively and accurately.
7. **Limit Switches**: Limit switches are crucial safety components in the 6-DOF serial chain manipulator. These switches detect the physical limits of each joint's range of motion, preventing over-extension and potential damage. The U-shaped optical limit switches use an infrared beam to detect interruptions, while mechanical limit switches rely on physical contact to trigger a response. When a joint reaches its predefined limit, the corresponding switch sends a signal to the controller, halting motor movement to protect the manipulator and ensure precise and safe operation. Mechanical limit switch is employed on the prismatic joint and U-shaped limit switches are mounted on the revolute joints.

7.2 Microprocessor Integration

The hardware architecture is built around STM32F103C8T6 microprocessor. In total, seven STM32F103C8T6 microprocessors were employed to ensure precise and reliable control of the manipulator's movements. One microprocessor serves

as the master controller, responsible for sending instructions and coordinating the overall operation of the system. The remaining six microprocessors function as slave controllers, each dedicated to controlling one of the six joints of the manipulator. This distributed control approach allows for specialized and efficient management of each joint's movements, enhancing the manipulator's performance and responsiveness. Communication between the master and slave microprocessors is facilitated by the CAN (Controller Area Network) bus protocol, which ensures real-time data exchange and synchronization across all components.

7.2.1 Master Controller:

The master controller is an STM32F103C8T6 microprocessor, which plays a crucial role in computing the inverse kinematics of the robot. This complex calculation involves determining the required angles for all six joints to achieve a desired end-effector position and orientation in space. Once the inverse kinematics computations are completed, the master controller converts these angular positions into corresponding encoder counts for each joint. These encoder counts are then transmitted to the respective slave microprocessors via the CAN (Controller Area Network) bus protocol. This method ensures precise and synchronized control of the manipulator's joints, allowing for smooth and accurate movements. By offloading the kinematics calculations to the master controller, each slave microprocessor can focus on executing the received commands, thereby enhancing the overall efficiency and performance of the robotic system.

7.2.2 Slave Controllers:

The slave controllers, each an STM32F103C8T6 microprocessor, form the backbone of the 6-DOF serial chain manipulator's joint control system. These controllers rely on the CAN (Controller Area Network) bus protocol as their primary means of communication with the master controller. Upon receiving the encoder counts from the master, each slave controller interprets these counts to determine

the precise movements needed for its respective joint. The slave controller then commands the motor connected to its joint to rotate by the specified encoder counts, ensuring accurate positioning. This coordinated effort allows each joint to move precisely as dictated by the master controller's inverse kinematics calculations, thereby enabling the manipulator to achieve the desired end-effector position and orientation with high precision. This decentralized control strategy not only improves efficiency but also ensures reliable and synchronized operation across all six joints of the robot.

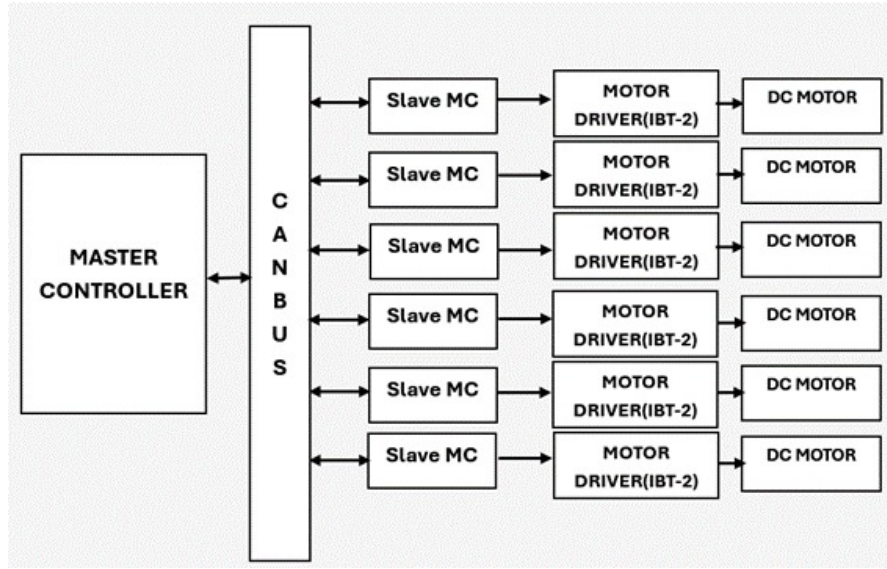


Figure 7.1: Overall Hardware circuit

7.3 Motor Control

In the 6-DOF serial chain manipulator, precise motor control is achieved through the integration of IBT-2 motor drivers and encoders. Each joint's movement is directed by a slave STM32F103C8T6 microprocessor, which receives target encoder counts from the master controller via the CAN bus protocol. The IBT-2 motor driver receives PWM signals from the slave controller, which dictate the speed and direction of the DC motor. Encoders attached to each motor provide real-time feedback on the joint's position by generating pulses as the motor rotates. The slave controller continuously monitors these pulses to compare the current

position with the target position, adjusting the motor's operation accordingly to achieve accurate and smooth joint movements. This closed-loop control system ensures that each joint reaches and maintains the desired position, enabling precise manipulation of the end-effector.

Chapter 8

Conclusion

8.1 Summary

This project focused on the comprehensive design and development of a control system for a 6 DOF (Degrees of Freedom) robotic arm, with the objective of enhancing end-effector control for various industrial applications. The initial phase involved designing and assembling the robot parts and selecting suitable components for the control system's hardware, including detailed circuit design. A robust communication protocol was established between the master and slave units using CANbus (Controller Area Network bus) communication, ensuring reliable and efficient data transfer within the system.

To enable the robotic arm to perform precise and efficient pick-and-place operations, sophisticated algorithms for trajectory planning were developed. Before implementation on physical hardware, the control system and the trajectory planning algorithms were tested and validated through simulations using V-REP software. After simulation testing, the control system was integrated with the physical robotic arm. Hardware tests were conducted to verify the performance of the integrated system. These tests demonstrated the system's capability to execute precise and efficient pick-and-place operations, validating the effectiveness of the developed control algorithms and the robustness of the communication protocols. Overall, the project successfully achieved its objective of enhancing end-effector

control for industrial applications through a well-designed and thoroughly tested control system for a 6 DOF robotic arm.

8.2 Future Work

Following are some suggestion for future work on this project.

- **Enhanced Autonomous Capabilities:** Refining algorithms for perception and decision-making to handle more complex environments using machine vision and synchronizing it with a virtual environment.
- **Teleoperation:** Developing more intuitive interfaces and controls to enhance teleoperation accessibility.
- **Advanced Trajectory Planning:** Further improving path planning algorithms for more complex tasks and better obstacle avoidance.
- **Application-Specific Customization:** Tailoring the system for specific industries such as collaborative robotics or specialized manufacturing processes.

8.3 Sustainable Development Goals

During our project, Design and Development of a Supervised Teleoperation Control System for a 6 DOF Serial Chain Manipulator, we aligned with and contributed to several Sustainable Development Goals (SDGs):

- **SDG 3: Good Health and Well-being** Our system's potential applications in medical robotics, particularly in surgical procedures, enhance the precision and effectiveness of minimally invasive surgeries, contributing to improved health outcomes.
- **SDG 8: Decent Work and Economic Growth** By improving productivity and safety in manufacturing and assembly tasks, our control system

reduces the need for human intervention in hazardous environments, supporting safer working conditions and sustainable industrial growth.

- **SDG 9: Industry, Innovation, and Infrastructure** Our work advances robotics and automation technologies, playing a crucial role in modernizing industries and infrastructure through innovative and resilient control systems for robotic manipulators.
- **SDG 12: Responsible Consumption and Production** By enhancing the precision and control of robotic systems, our project helps to reduce waste and optimize resource use, promoting sustainable production patterns.
- **SDG 13: Climate Action** Our teleoperated robotic systems can be deployed in hazardous or inaccessible environments, aiding in environmental monitoring and disaster response, and addressing challenges related to climate change.
- **SDG 4: Quality Education** Our thesis supports the development of educational tools and platforms, enabling students and researchers to engage with advanced robotics technologies, thus contributing to lifelong learning and technical skill development.

References

1. S. H. Mark W. Spong and M. Vidyasagar, Robot Dynamics and Control, Wiley, 2004.
2. R. J. Schilling, Fundamentals of Robotics Analysis and Control, Prentice-Hall, 2003.
3. Y. Jadeja and B. Pandya, "Design and development of 5-dof robotic arm manipulators," International Journal of Scientific Technology Research, vol. 8, no. 1, pp. 2158-2167, 2019.
4. A. M. Muhammad Bilal, Muhammad Osama Khan and N. Ali, "Design and control of 6 dof robotic manipulator," vol. 1, no. 1, p. 04, 2018.
5. T. H. C. Pollette, "How robots work." 2020. [Online]. Available: Last accessed 19 February 2023.
6. S. H. Mark W. Spong and M. Vidyasagar, "Forward kinematics: The Denavit-Hartenberg convention," in Robot Dynamics and Control, 2nd ed., Wiley Publishing, 2004, pp. 61-72.
7. R. J. Schilling, "A five articulated robot (rihno xr-3)," in Fundamentals of Robotics Analysis and Control, 2nd ed., Prentice-Hall, 2003, pp. 62-68.
8. R. J. Schilling, "Inverse kinematics of a five-axis articulated robot (rihno xr-3)," in Fundamentals of Robotics Analysis and Control, 2nd ed., Prentice-Hall, 2003, pp. 90-96.
9. C. A. M. Sanchez, Six Degree of Freedom Robot Manipulator, PhD thesis, University of Saskatchewan, 2012.
10. J. Teel, "Introduction to programming stm32 arm cortex-m 32-bit micro-controllers," 2022. [Online]. Available: Last accessed 19 February 2023.

11. R. Teja, "Getting started with esp32 introduction to esp32," 2021. [Online]. Available: Last accessed 19 February 2023.
12. Q. G. Zhijie Lin, Tielu Wang and Y. Liu, "Design of robot platform based on can bus," International Conference on Electrical and Control Engineering, no. 1, 2011.
13. N. M. E.-R. Ebrahim Abd El-Hamid Mohamed Ramadan, Mohammad El-Bardini and M. A. Fkirin, "Embedded system based on a real time fuzzy motor speed controller," Ain Shams Engineering Journal, vol. 5, no. 1, pp. 399-409, 2019.
14. M. A. Hamza and M. A. Amjad, Use PID controller to develop a DC motor based speed control system, PhD thesis, COMSATS University Islamabad, 2020.

About the Author



Muhammad Sheraz was born on June 11, 2002, in Peshawar, KPK Province, Pakistan. He did his matriculation and F.Sc - Pre-Engineering from Fazaia Degree College Risalpur, securing high merits. The author joined the Pakistan Institute of Engineering & Applied Sciences, Islamabad as a Bachelor's student of Electrical Engineering. His areas of interest are Embedded Control Systems, Robotics & Automation, and Computational Intelligence in Engineering Applications.

Email: Dsshaizy@gmail.com

About the Author



Muhammad Huzaifa has completed his Bachelor of Science in Electrical Engineering from the Pakistan Institute of Engineering and Applied Sciences, where he developed a strong foundation in electrical engineering with a specialized focus on robotic systems and embedded control design. His career is centered on advancing automation and robotics, with an emphasis on creating innovative control systems for complex robotic applications. He is proficient in designing and implementing embedded systems, real-time control algorithms, and advanced robotics technologies. This thesis showcases his technical expertise and commitment to addressing critical challenges in modern robotics and automation. Muhammad Huzaifa is dedicated to leveraging his skills and knowledge to contribute to the future of robotics and embedded systems, driving innovation in the field of automation.

Email: huzaifazahid888@gmail.com